

## 5. Schaltwerke

### 5.1 Sequentielle Logik

#### Definition (Schaltwerk SW):

Ein Schaltwerk (oder sequentielle Logik, engl. sequential logic) ist eine digitale Schaltung, in der die Ausgangskombination nicht nur abhängt von der Eingangsbelegung, sondern auch von einem inneren Zustand. Dieser wiederum ist abhängig von den Eingangsbelegungen der Vergangenheit (Vorgeschichte).

Ein SW hat also ein „Gedächtnis“, das durch Speicherglieder für den inneren Zustand realisiert wird.

kombinatorische Logik:  $A_j(t_n) = f[E_i(t_n)]$



sequentielle Logik:

$$A_j(t_n) = f[E_i(t_n), E_i(t_{n-1}), \dots, E_i(t_0)]$$

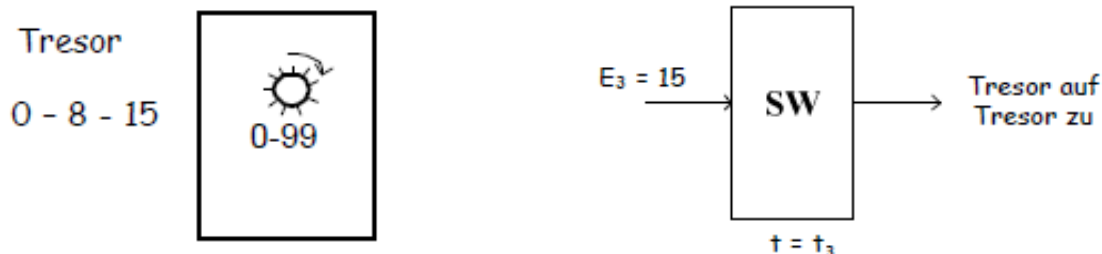


Vorgeschichte  
→ interner Zustand des  
Schaltwerks.

$Z_n(t_n)$

$$A_j(t_n) = f[E_i(t_n), Z_k(t_n)]$$

Der innere Zustand muss aber nicht alle Details der Vorgeschichte widerspiegeln, sondern nur die „relevante“ Information. Dies verdeutlicht das folgende Beispiel:

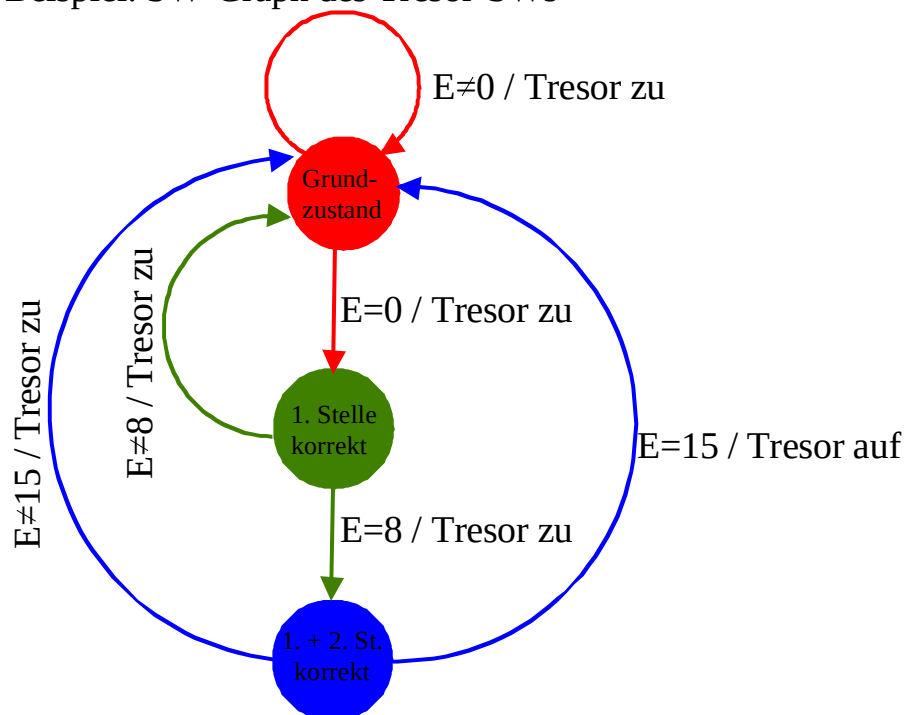


Zum Zeitpunkt  $t_3$  ist nur wichtig: Waren zu den Zeitpunkten  $t_1$  und  $t_2$  die Werte 0 und 8 eingestellt (also die richtige Kombination), oder irgendeine andere, falsche Kombination. Welche der 9999 falschen Kombinationen vorlag, ist für das Verhalten des SWs irrelevant.

## 5.2 Formale Beschreibung von Schaltwerken

Eine grafische Darstellung des Schaltwerksverhaltens ist der Schaltwerksgraph (Zustandsdiagramm, Kreisdiagramm). Hierbei werden die inneren Zustände des Schaltwerks durch Knoten (Kreise) dargestellt, die Zustandsübergänge durch Kanten (Linien zwischen den Kreisen). Die Beschriftung an den Kanten ist Eingabe / Ausgabe: Wenn die Eingabe auftritt, wird die entsprechende Kante durchlaufen und die notierte Ausgabe ist relevant. Von jedem Knoten muss für jede Eingabe eine Kante ausgehen.

Beispiel: SW-Graph des Tresor-SWs



Im Folgenden soll als weiteres Beispiel ein Geldwechselautomat betrachtet werden, dessen Funktion wie folgt beschrieben werden kann:

Der Automat soll Euro-Münzen (1EUR oder 2EUR) in 20ct-Stücke wechseln. Es können maximal zwei Euro gewechselt werden. Als Eingabe werden nur 1EUR und 2EUR Münzen akzeptiert. Bei anderen Münzen oder Überbezahlung wird alles zurückgegeben und der Vorgang abgebrochen. Das Wechseln wird durch eine Taste „WT“ angefordert.

Es gibt also 4 mögliche Eingaben und 4 mögliche Ausgaben:

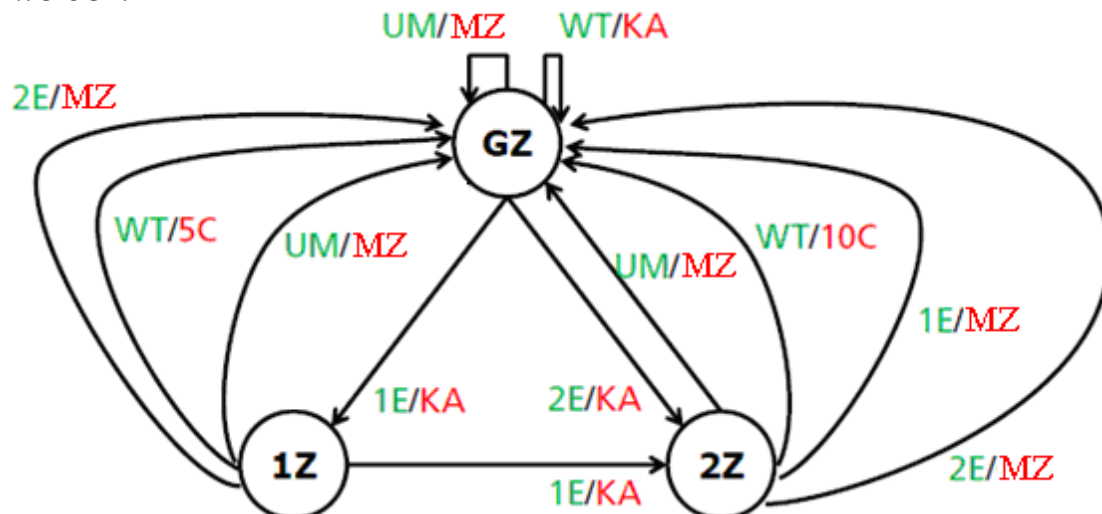
- Eingabemenge: {1E, 2E, UM, WT}
  - 1E: 1EUR-Münze
  - 2E: 2EUR-Münze
  - UM: ungültige Münze
  - WT: Wechseltaste gedrückt

- **Ausgabemenge:** {KA, MZ, 5C, 10C}  
 KA: Keine Aktion  
 MZ: Münzen zurück  
 5C: 5 mal 20ct  
 10C: 10 mal 20ct

Aus der Aufgabenstellung folgt weiterhin, dass 3 innere Zustände benötigt werden:

- **Zustandsmenge:** {GZ, 1Z, 2Z}  
 GZ: Grundzustand – kein Guthaben  
 1Z: Guthaben: ein Euro  
 2Z: Guthaben: zwei Euro

Das korrekte Verhalten kann mit dem folgenden SW-Graphen beschrieben werden:



Eine äquivalente, nicht-grafische Beschreibung liefert die Folgezustandstabelle, die für unser Beispiel wie folgt lautet:

| Zustand | Eingabe | Ausgabe | Folgezustand |
|---------|---------|---------|--------------|
| GZ      | 1E      | KA      | 1Z           |
| GZ      | 2E      | KA      | 2Z           |
| GZ      | UM      | MZ      | GZ           |
| GZ      | WT      | KA      | GZ           |
| 1Z      | 1E      | KA      | 2Z           |
| 1Z      | 2E      | MZ      | GZ           |
| 1Z      | UM      | MZ      | GZ           |
| 1Z      | WT      | 5C      | GZ           |
| 2Z      | 1E      | MZ      | GZ           |
| 2Z      | 2E      | MZ      | GZ           |
| 2Z      | UM      | MZ      | GZ           |
| 2Z      | WT      | 10C     | GZ           |

Vor der Realisierung als Digitalschaltung muss eine Kodierung der Eingaben, Ausgaben und der Zustände erfolgen. Im Beispiel werden zwei Eingabevariablen  $e_1$  und  $e_0$  (für 4 Eingaben), zwei Ausgabevariablen  $a_1$  und  $a_0$  (für 4 Ausgaben) und zwei **Zustandsvariablen**  $q_1$  und  $q_0$  (für 3 Zustände – eine Kombination der Zustandsvariablen bleibt ungenutzt) benötigt.

Die Kodierung von Ein- und Ausgabe sei wie folgt festgelegt:

| Eingabe | $e_1$ $e_0$ | Ausgabe | $a_1$ $a_0$ |
|---------|-------------|---------|-------------|
| UM      | 0 0         | KA      | 0 0         |
| 1E      | 0 1         | 5C      | 0 1         |
| 2E      | 1 0         | 10C     | 1 0         |
| WT      | 1 1         | MZ      | 1 1         |

Die Zustandskodierung kann beim Entwurf der Digitalschaltung frei gewählt werden. Die gewählte Zustandskodierung ist in der folgenden Tabelle dargestellt:

| Zustand       | $q_1$ $q_0$ |
|---------------|-------------|
| GZ            | 0 0         |
| 1Z            | 0 1         |
| 2Z            | 1 0         |
| Pseudozustand | 1 1         |

Sind alle Kodierungen festgelegt, kann die Folgezustandstabelle (oder direkt der SW-Graph) in die so genannte **Schaltwerkstabelle** überführt werden. Diese enthält auf der linken Seite Spalten für Eingabe- und Zustands**variablen**, auf der rechten Seite Spalten für Ausgabe- und Zustands**variablen** für den Folgezustand. Der Folgezustand wird durch das hochgestellte "+" gekennzeichnet. Die Reihenfolge der Spalten auf der linken und rechten Seite ist nicht festgelegt.

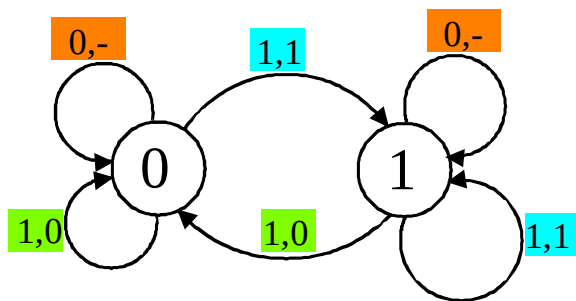
| $e_1$ $e_0$ | $q_1$ $q_0$ | $q_1^+$ $q_0^+$ | $a_1$ $a_0$ |
|-------------|-------------|-----------------|-------------|
| 0 0         | 0 0         | 0 0             | 1 1         |
| 0 0         | 0 1         | 0 0             | 1 1         |
| 0 0         | 1 0         | 0 0             | 1 1         |
| 0 0         | 1 1         | - -             | - -         |
| 0 1         | 0 0         | 0 1             | 0 0         |
| 0 1         | 0 1         | 1 0             | 0 0         |
| 0 1         | 1 0         | 0 0             | 1 1         |
| 0 1         | 1 1         | - -             | - -         |
| 1 0         | 0 0         | 1 0             | 0 0         |
| 1 0         | 0 1         | 0 0             | 1 1         |
| 1 0         | 1 0         | 0 0             | 1 1         |
| 1 0         | 1 1         | - -             | - -         |
| 1 1         | 0 0         | 0 0             | 0 0         |
| 1 1         | 0 1         | 0 0             | 0 1         |
| 1 1         | 1 0         | 0 0             | 1 0         |
| 1 1         | 1 1         | - -             | - -         |

Die Schaltungstabelle ist die Grundlage für den Entwurf einer Digitalschaltung, die das spezifizierte Steuerwerksverhalten realisiert. Bevor dieser Entwurfsvorgang systematisch dargestellt wird, werden in den nächsten beiden Abschnitten digitale Speicherglieder besprochen, die für die Speicherung der Zustandsvariablen benötigt werden.

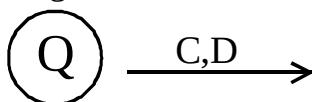
### 5.3 Schaltnetze mit Rückführungen

Die einfachsten Speicherglieder heißen Flip-Flops: Sie haben zwei Zustände ("Zweikipp") und daher eine Zustandsvariable, im Folgenden mit  $Q$  bezeichnet. Diese Zustandsvariable dient auch als Ausgangsvariable, weitere Ausgangsvariablen sind nicht vorhanden.

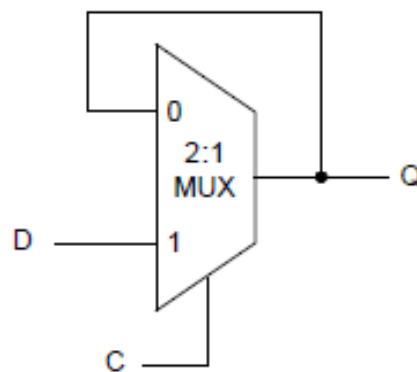
Die folgenden Abbildungen zeigen den SW-Graph und die SW-Tabelle eines solchen Speichergliedes und die Realisierung als Schaltnetz mit Rückführung:



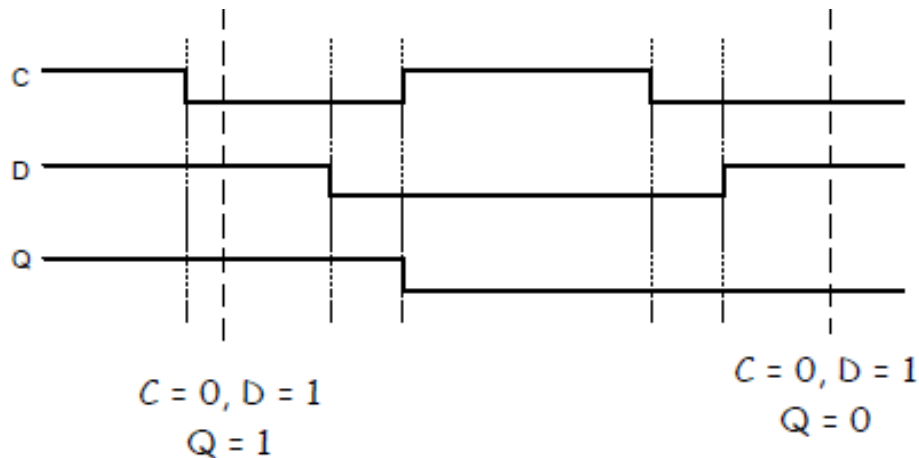
Legende:



| C | D | Q | Q <sup>+</sup> |            |
|---|---|---|----------------|------------|
| 0 | 0 | 0 | 0              | Speichern  |
| 0 | 0 | 1 | 1              | Speichern  |
| 0 | 1 | 0 | 0              | Speichern  |
| 0 | 1 | 1 | 1              | Speichern  |
| 1 | 0 | 0 | 0              | Rücksetzen |
| 1 | 0 | 1 | 0              | Rücksetzen |
| 1 | 1 | 0 | 1              | Setzen     |
| 1 | 1 | 1 | 1              | Setzen     |



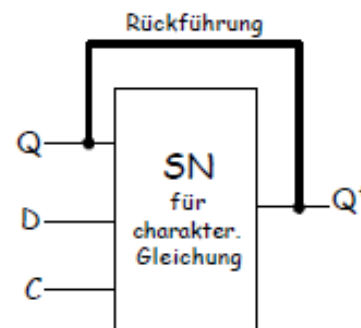
Dass eine solche einfache Schaltung mit Rückführung bereits SW-Verhalten aufweist, macht das folgende Zeitdiagramm (auch Impulsdiagramm genannt) deutlich:



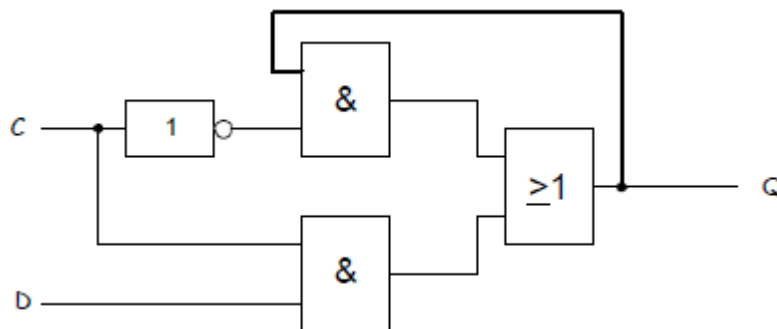
Ausgehend von der SW-Tabelle erhält man die Schaltung, wenn man zunächst ein SN für  $Q^+$  entwirft:

$$Q^+ = C D \vee \bar{C} Q$$

Dies ist die so genannte "charakterische Gleichung" des SWs. Durch die Rückführung von  $Q^+$  auf  $Q$  realisiert man dann das SW-Verhalten. Diese Vorgehensweise ist der asynchrone Entwurf eines SWs:



Natürlich kann man statt des Multiplexers auch einzelne Gatter verwenden:

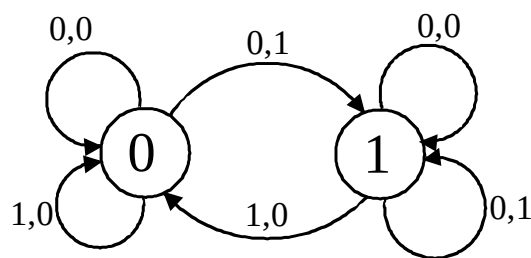


Hier kann man schon ahnen, dass der asynchrone Entwurf nicht ganz unproblematisch ist. Tatsächlich kann es schon bei der obigen Schaltung auf Grund von ungünstigen Gatterlaufzeiten zu einem Fehlverhalten kommen. Wir daher im nächsten Abschnitt einen anderen, sicheren Entwurf dieses Speichergliedes (Name: taktzustandsgesteuertes D-Flip-Flop) betrachten.

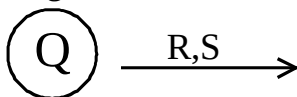
Zunächst wird der Entwurf des asynchronen RS-Flip-Flops besprochen. Diese Schaltung ist einfacher und hat einen unkritischen Rückführungspfad, so dass ein sicheres Funktionieren unabhängig von den Gatterlaufzeiten sicher gestellt ist.

## Asynchrones RS-Flip-Flop

Der SW-Graph und die Funktionstabelle des asynchronen RS-Flip-Flops sind in der folgenden Abbildung dargestellt. Die Funktionstabelle ist eine weitere Beschreibungsmöglichkeit für FFs (gebräuchlich in Datenblättern): der Übergang in den Zustand  $Q=0$  wird als Rücksetzen bezeichnet, der Übergang in den Zustand  $Q=1$  als Setzen. Das Verbleiben im aktuellen Zustand heißt Speichern.



Legende:



$R,S=1,1$  ist nicht definiert

| R | S | $Q^+$ | Funktion   |
|---|---|-------|------------|
| 0 | 0 | Q     | Speichern  |
| 0 | 1 | 1     | Setzen     |
| 1 | 0 | 0     | Rücksetzen |
| 1 | 1 | -     | Verboten   |

Daraus ergibt sich die folgende SW-Tabelle:

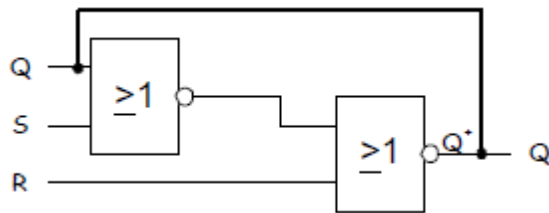
| R | S | Q | $Q^+$ |
|---|---|---|-------|
| 0 | 0 | 0 | 0     |
| 0 | 0 | 1 | 1     |
| 0 | 1 | 0 | 1     |
| 0 | 1 | 1 | 1     |
| 1 | 0 | 0 | 0     |
| 1 | 0 | 1 | 0     |
| 1 | 1 | 0 | -     |
| 1 | 1 | 1 | -     |

Beim asynchronen Entwurf ist nun das SN für die charakteristische Gleichung  $Q^+ = f(R,S,Q)$  zu entwerfen:

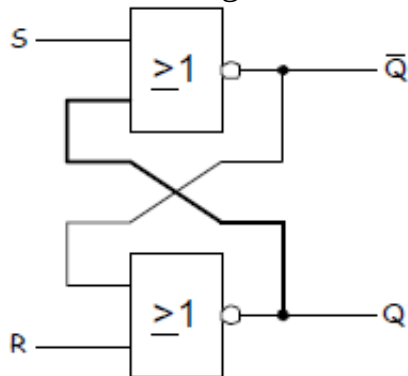
|     |     |   |   |   |
|-----|-----|---|---|---|
|     | $Q$ |   |   |   |
|     | 0   | 1 | 0 | 0 |
| $S$ | 1   | 1 | - | - |
|     |     |   |   |   |
|     | $R$ |   |   |   |

$$\begin{aligned}
 Q^+ &= \bar{R} Q \vee \bar{R} S \\
 &= \bar{R} (Q \vee S) \\
 &= \overline{\overline{\bar{R} (Q \vee S)}} \\
 &= \overline{\overline{R} \wedge \overline{Q \vee S}} \\
 &= R \vee Q \vee S
 \end{aligned}$$

Die Rückführung von  $Q^+$  auf  $Q$  ergibt dann die folgende Schaltung:



Oder etwas umgezeichnet:

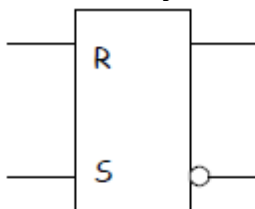


Dieses Speicherglied (das unabhängig von den Gatterlaufzeiten sicher funktioniert) wird nun im Folgenden eingesetzt, um weitere Speicherglieder zu entwerfen. Dafür ist noch eine weitere Beschreibungsform nötig, die so genannte Synthesetabelle:

| Q | $Q^+$ | R | S |
|---|-------|---|---|
| 0 | 0     | - | 0 |
| 0 | 1     | 0 | 1 |
| 1 | 0     | 1 | 0 |
| 1 | 1     | 0 | - |

Sie ist so aufgebaut, dass auf der linken Seite alle möglichen Zustandsübergänge aufgelistet sind; auf der rechten Seite ist eingetragen, wie das Flip-Flop an den R- und S-Eingängen anzusteuern ist, um den jeweiligen Zustandsübergang zu bewirken. Es ist sichergestellt, dass die verbotene Ansteuerung  $R,S=1,1$  nicht vorkommt.

Das Schaltsymbol des asynchronen RS-Flip-Flops ist das folgende:





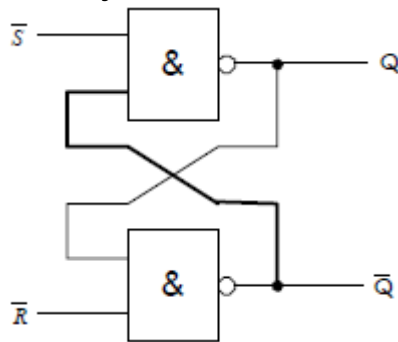
### Variante: $\overline{R}\overline{S}$ -Flip-Flop

Bei dieser erfolgt die Ansteuerung des Rücksetz-Eingangs  $R$  und des Setzeingangs  $S$  0-aktiv (active low), d. h. die jeweilige Funktion wird durch den Logikzustand 0 bewirkt. Bei  $\overline{R} = 0$  wird das Flip-Flop zurückgesetzt, bei  $\overline{S} = 0$  gesetzt. Beide Eingänge dürfen nicht gleichzeitig 0 sein.

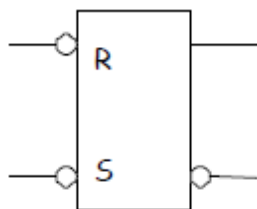
Die Funktionstabelle sieht wie folgt aus:

| $\overline{R}$ | $\overline{S}$ | $Q^+$ | Funktion   |
|----------------|----------------|-------|------------|
| 0              | 0              | -     | Verboten   |
| 0              | 1              | 0     | Rücksetzen |
| 1              | 0              | 1     | Setzen     |
| 1              | 1              | $Q$   | Speichern  |

Der asynchrone Entwurf führt zu der folgenden Schaltung:



Die nächste Abbildung zeigt das Schaltsymbol und die Synthesetabelle:



| $Q$ | $Q^+$ | $\overline{R}$ | $\overline{S}$ |
|-----|-------|----------------|----------------|
| 0   | 0     | -              | 1              |
| 0   | 1     | 1              | 0              |
| 1   | 0     | 0              | 1              |
| 1   | 1     | 1              | -              |

## 5.4 Speicherglieder mit Taktsteuerung

### 5.4.1 Das taktzustandsgesteuerte D-Flip-Flop

Das aus dem vorigen Abschnitt bereits bekannte taktzustandsgesteuerte D-Flip-Flop soll nun mit Hilfe eines RS-Flip-Flops neu entworfen werden: die Lösung besteht dann aus einem RS-Flip-Flop und geeigneten Ansteuerschaltnetzen für den R- und S-Eingang.

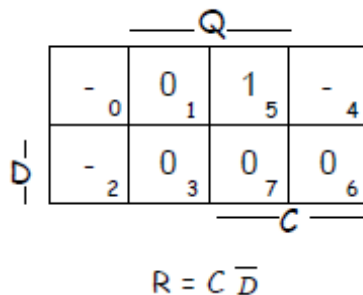
Ausgangspunkt für den Entwurf ist wieder die Schaltwerkstabelle. Diese wird ergänzt um Spalten für die Ansteuerung der Eingänge R und S (Ansteuertabelle). Die passenden Logikwerte für die Ansteuerung findet man, wenn man in jeder

Zeile den Übergang  $Q/Q^+$  betrachtet und die dafür benötigte Ansteuerung aus der Synthesetabelle des RS-Flip-Flops abliest:

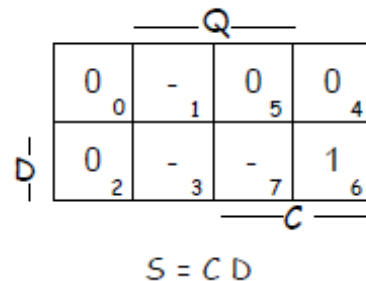
| C | D | Q | Q <sup>+</sup> | R | S |
|---|---|---|----------------|---|---|
| 0 | 0 | 0 | 0              | - | 0 |
| 0 | 0 | 1 | 1              | 0 | - |
| 0 | 1 | 0 | 0              | - | 0 |
| 0 | 1 | 1 | 1              | 0 | - |
| 1 | 0 | 0 | 0              | - | 0 |
| 1 | 0 | 1 | 0              | 1 | 0 |
| 1 | 1 | 0 | 1              | 0 | 1 |
| 1 | 1 | 1 | 1              | 0 | - |

Die Schaltnetze für die Ansteuerfunktionen für R und S können jetzt in bekannter Weise entworfen werden:

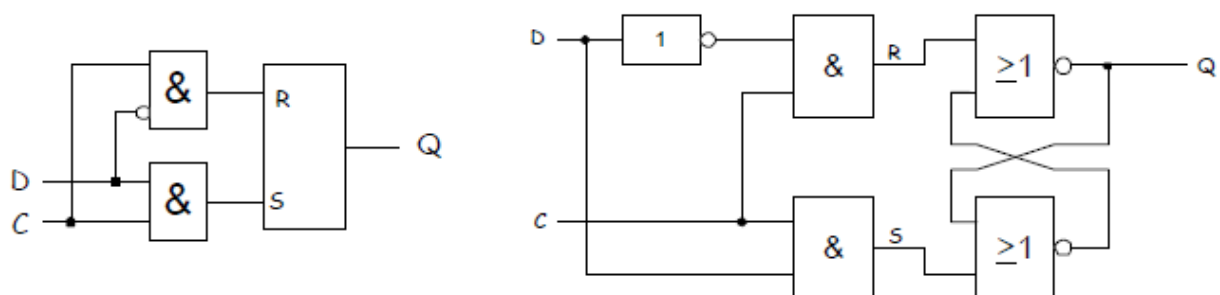
R



S



Damit erhält man die folgende (sichere) Schaltung für das taktzustandsgesteuerte D-Flip-Flop:



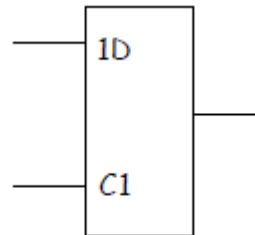
Das Eingangssignal  $C$  ist insofern ein besonderes Signal, das bei  $C=0$  keine Zustandsübergänge möglich sind: Unabhängig vom Logikwert am Eingang  $D$  ("Dateneingang") bleibt der alte Zustand gespeichert.

Ein Eingangssignal, das festlegt, wann Zustandsänderungen überhaupt möglich sind, heißt **Takt**. Verboten der Takt Zustandsänderungen, so sind die anderen Eingangssignale irrelevant. Die anderen Eingangssignale sind also vom Takt abhängig.

Das Taktsignal kann, wie im obigen Fall, Zeiträume ( $C = 1$ ) definieren, an denen Zustandsübergänge möglich sind. Man spricht dann von einer Taktzustandssteuerung.

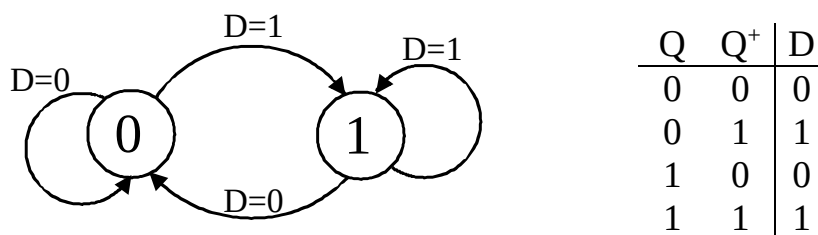
Bei der Taktflankensteuerung sind es nur einzelne Zeitpunkte, bei denen Zustandsübergänge möglich sind. Üblicherweise werden diese Zeitpunkte durch den Übergang des Taktsignals von 0 nach 1 (steigende Taktflanke:  $C = \uparrow$ ) definiert.

Beim Schaltsymbol des taktzustands-gesteuerten D-Flip-Flops wird der Takteingang durch ein C und eine nachgestellte Ziffer gekennzeichnet. Der davon abhängige D-Eingang hat die gleiche Ziffer vorangestellt.



Ist bei einem Schaltwerk (einfaches Speicherglied oder komplexes SW) ein Taktsignal vorhanden, braucht dieses beim Entwurf nicht besonders berücksichtigt werden: Es kann schon bei der formalen Beschreibung weggelassen werden. Natürlich muss man dann implizit wissen, dass eine Taktsteuerung (und welche Art der Taktsteuerung) vorhanden ist.

Mit dieser Zusatzinformation ist folgende Darstellung des SW-Graphen und der Synthesetabelle des D-Flip-Flops möglich:



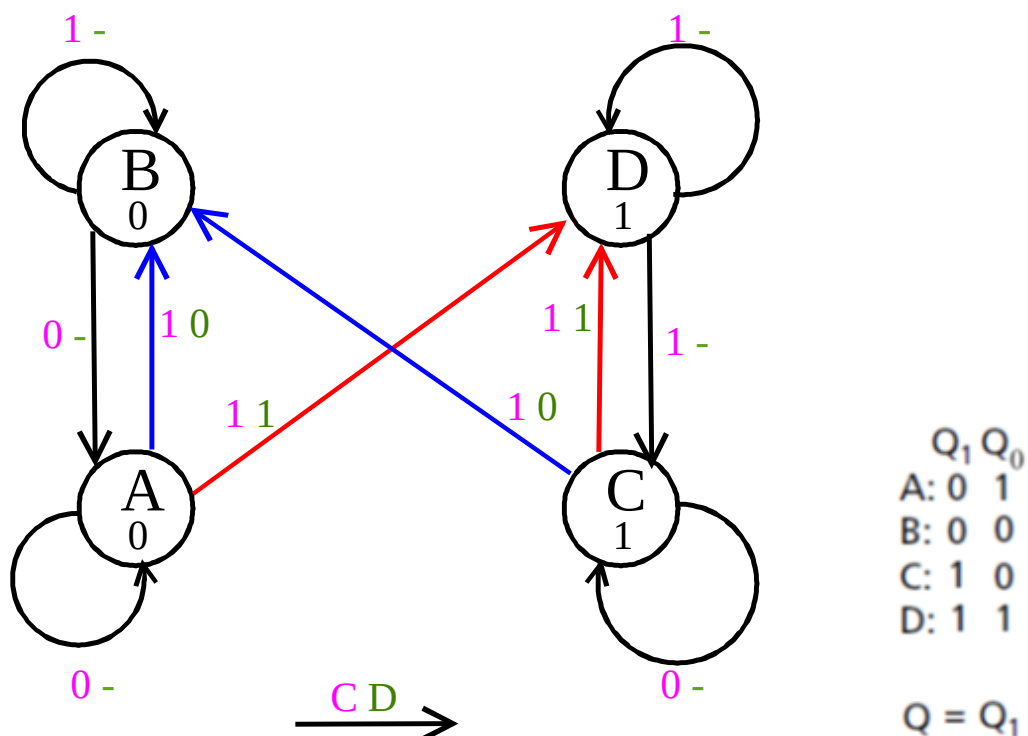
Sinnvoll ist diese Darstellung nur, wenn man noch um das Vorhandensein eines Taktsignals weiß (ansonsten hätte man ein SN-Verhalten: aus  $D=0 \rightarrow Q=0$  und aus  $D=1 \rightarrow Q=1$ ). Dabei kann es sich um eine Taktzustandssteuerung oder um eine Taktflankensteuerung (siehe nächsten Abschnitt) handeln.

### 5.4.2 Das taktf flankengesteuerte D-Flip-Flop

In diesem Abschnitt soll das taktf flankengesteuerte D-Flip-Flop entworfen werden, bei dem nur beim 0/1-Übergang des Taktsignals C eine Zustandsänderung möglich ist.

Nach dem Entwurf gilt der obige SW-Graph mit der Zusatzinformation „Taktflankensteuerung durch das Eingangssignal C“. Für den Entwurf, den wir mit Hilfe zweier RS-Flip-Flops durchführen werden, müssen wir den SW-Graphen mit beiden Eingangssignalen C und D spezifizieren:

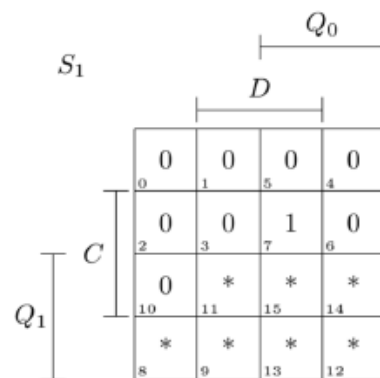
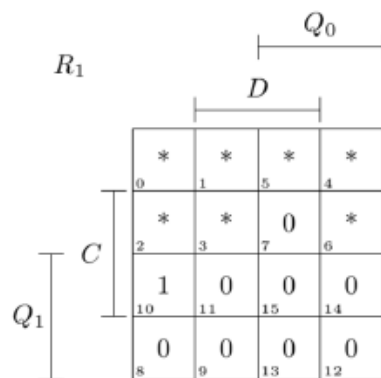
Es zeigt sich, dass jetzt 4 „innere“ Zustände benötigt werden: In zwei Zuständen (A und B, linke Seite) ist das Flip-Flop rückgesetzt ( $Q=0$ ), in den beiden anderen Zuständen (C und D, rechte Seite) ist das Flip-Flop gesetzt ( $Q=1$ ). Die „unteren“ Zustände A und C werden erreicht, wenn  $C=0$  geworden ist. In diesen beiden Zuständen ist ein Wechsel der „Seite“ möglich, wenn wieder  $C=1$  wird. Abhängig vom Eingangssignal D werden dann die blauen ( $D=0$ ) oder roten ( $D=1$ ) Kanten durchlaufen.



Die SW-Tabelle, ergänzt um die Ansteuerfunktionen für 2 RS-Flip-Flops (Ansteuertabelle) lautet wie folgt:

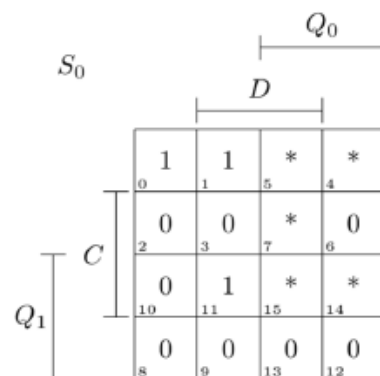
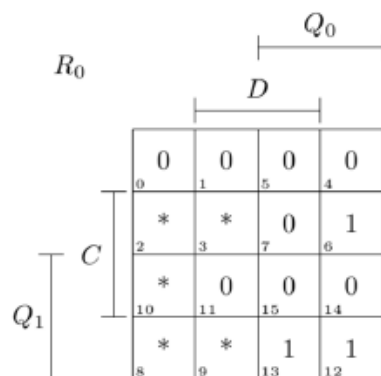
| $Q_1$ | $Q_0$ | $C$ | $D$ | $Q_1^+$ | $Q_0^+$ | $R_1$ | $S_1$ | $R_0$ | $S_0$ |
|-------|-------|-----|-----|---------|---------|-------|-------|-------|-------|
| 0     | 0     | 0   | -   | 0       | 1       | -     | 0     | 0     | 1     |
| 0     | 0     | 1   | -   | 0       | 0       | -     | 0     | -     | 0     |
| 0     | 1     | 0   | -   | 0       | 1       | -     | 0     | 0     | -     |
| 0     | 1     | 1   | 0   | 0       | 0       | -     | 0     | 1     | 0     |
| 0     | 1     | 1   | 1   | 1       | 1       | 0     | 1     | 0     | -     |
| 1     | 0     | 0   | -   | 1       | 0       | 0     | -     | -     | 0     |
| 1     | 0     | 1   | 0   | 0       | 0       | 1     | 0     | -     | 0     |
| 1     | 0     | 1   | 1   | 1       | 1       | 0     | -     | 0     | 1     |
| 1     | 1     | 0   | -   | 1       | 0       | 0     | -     | 1     | 0     |
| 1     | 1     | 1   | -   | 1       | 1       | 0     | -     | 0     | -     |

Die Ansteuerfunktionen können nun im KV-Diagramm entworfen werden:



$$R_1 = C \bar{D} \bar{Q}_0$$

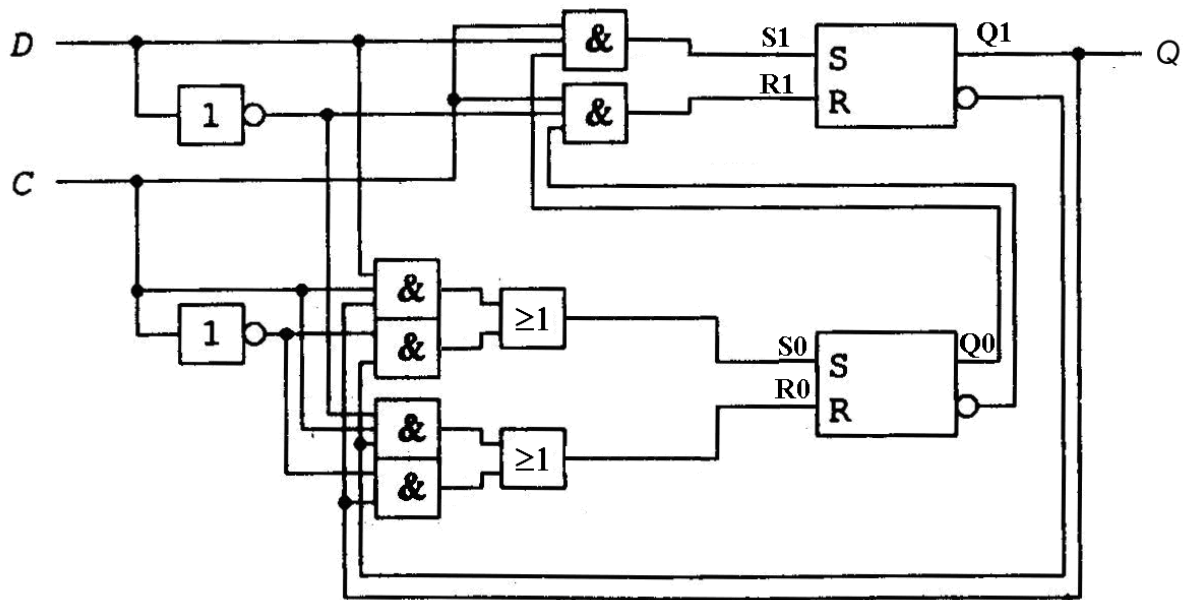
$$S_1 = C D Q_0$$



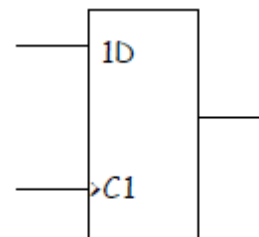
$$R_0 = \bar{C} Q_1 \vee C \bar{D} \bar{Q}_1$$

$$S_0 = \bar{C} \bar{Q}_1 \vee C D Q_1$$

Damit erhält man die folgende Gatterschaltung:

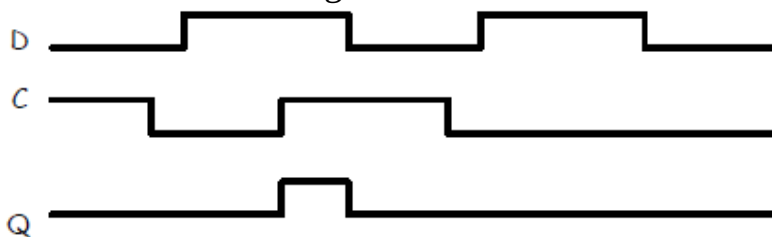


Das Schaltsymbol des taktflankengesteuerten D-Flip-Flops unterscheidet sich von dem des taktzustandsgesteuerten D-Flip-Flops durch die Dreiecksmarkierung am Takteingang:

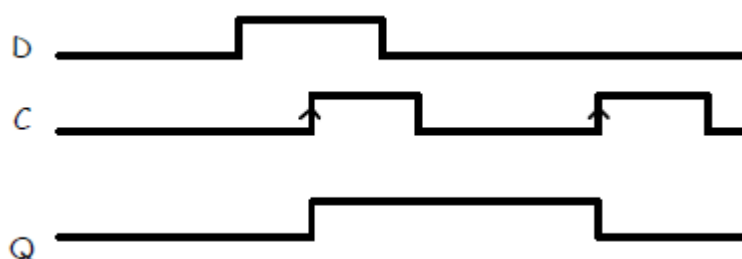


Die folgenden beiden Zeitdiagramme verdeutlichen nochmals das unterschiedliche Verhalten bei Taktzustands- bzw. bei Taktflankensteuerung:

Taktzustandssteuerung:



Taktflankensteuerung:

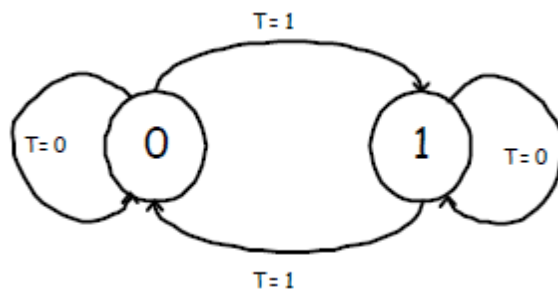


Die Funktionstabelle des taktflankengesteuerten D-Flip-Flops wird wie folgt spezifiziert:

| C | D | Q* | Funktion   |
|---|---|----|------------|
| 0 | - | Q  | Speichern  |
| 1 | - | Q  | Speichern  |
| ↓ | - | Q  | Speichern  |
| ↑ | 0 | 0  | Rücksetzen |
| ↑ | 1 | 1  | Setzen     |

### 5.4.3 Das T-Flip-Flop

Beim Toggle-Flip-Flop (T-FF) wird über den Eingang T zwischen den Funktionen Speichern und "Toggeln" ausgewählt: Toggeln bezeichnet den Übergang in den anderen Zustand:



Wegen der Funktion Toggeln ( $T=1$ ) ist eine Taktflankensteuerung erforderlich, sonst würden laufzeitabhängige Schwingungen auftreten (unvorhersehbares Verhalten!).

Die Funktionstabelle lautet wie folgt:

| T | C | Q*        | Funktion  |
|---|---|-----------|-----------|
| - | 0 | Q         | Speichern |
| - | 1 | Q         | Speichern |
| - | ↓ | Q         | Speichern |
| 0 | ↑ | Q         | Speichern |
| 1 | ↑ | $\bar{Q}$ | Toggeln   |

Den Entwurf könnte man wie beim taktflankengesteuerten D-Flip-Flop durchführen: Ausgangspunkt wäre dann ein SW-Graph (4 innere Zustände), der beide Eingangssignale T und C berücksichtigt.

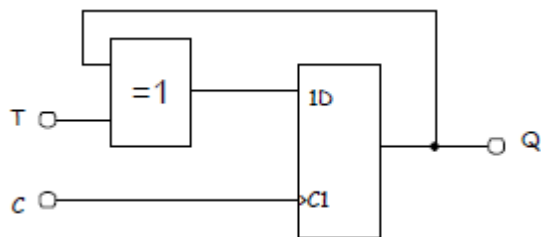
Einfacher ist es aber, das taktflankengesteuerte D-Flip-Flop als Speicherglied zu benutzen. Beim Entwurf braucht dann der Takt nicht berücksichtigt zu werden. Ausgehend vom obigen SW-Graphen (ohne Takt) erhält man die SW-Tabelle des T-Flip-Flops und kann diese um die Ansteuerfunktion D für ein D-Flip-Flop ergänzen (mit Hilfe der Synthesetabelle des D-Flip-Flops):

| T | Q | Q <sup>+</sup> | D |
|---|---|----------------|---|
| 0 | 0 | 0              | 0 |
| 0 | 1 | 1              | 1 |
| 1 | 0 | 1              | 1 |
| 1 | 1 | 0              | 0 |

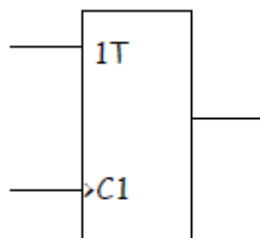
$$D = T \oplus Q$$

Ansteuerfunktion

Somit wird das T-Flip-Flop durch die folgende Schaltung realisiert:

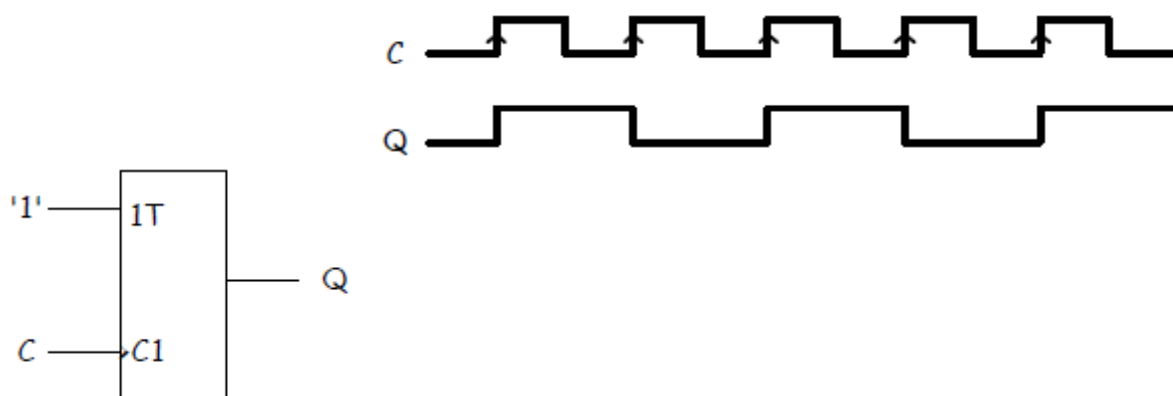


Die nächste Abbildung zeigt das Schaltsymbol des T-Flip-Flops und seine Synthesetabelle.



| Q | Q <sup>+</sup> | T |
|---|----------------|---|
| 0 | 0              | 0 |
| 0 | 1              | 1 |
| 1 | 0              | 1 |
| 1 | 1              | 0 |

Eine einfache Anwendung des T-Flip-Flops - neben der Anwendung als Speicherglied für Schaltwerke - ist eine Takteilerschaltung:





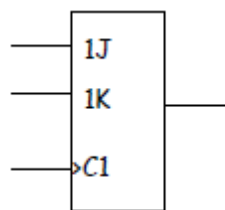
### 5.4.4 Das JK-Flip-Flop

Das JK-Flip-Flop mit den Ansteuereingängen J (jump) und K (kill) realisiert 4 Funktion gemäß der folgenden Funktionstabelle:

| J | K | C | Q <sup>+</sup> |
|---|---|---|----------------|
| - | - | 0 | Q              |
| - | - | 1 | Q              |
| - | - | ↓ | Q              |
| 0 | 0 | ↑ | Q              |
| 0 | 1 | ↑ | 0              |
| 1 | 0 | ↑ | 1              |
| 1 | 1 | ↑ | $\bar{Q}$      |

Wegen der Funktion „Toggeln“ ist auch bei diesem FF eine Taktflankensteuerung erforderlich. Der Entwurf kann wieder auf dieselbe Art und Weise wie beim T-FF erfolgen, dass der Takt weggelassen wird und die Ansteuerfunktionen für ein bereits zur Verfügung stehendes taktflankengesteuertes Flip-Flop (D-FF oder T-FF) ermittelt werden ("Flip-Flop-Transformation").

Schaltsymbol und Synthesetabelle lauten wie folgt:

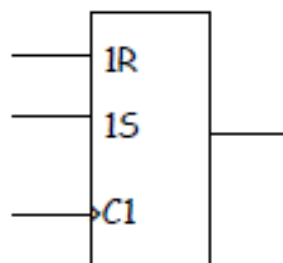


| Q | Q <sup>+</sup> | J | K |
|---|----------------|---|---|
| 0 | 0              | 0 | - |
| 0 | 1              | 1 | - |
| 1 | 0              | - | 1 |
| 1 | 1              | - | 0 |

### 5.4.5 RS-Flip-Flops mit Taktsteuerung

Das bereits in 5.3 besprochene RS-Flip-Flop (mit Setz- und Rücksetzeingang) ist auch mit Taktsteuerung realisierbar: Es gibt sowohl ein taktzustandsgesteuertes RS-FF als auch ein taktflankengesteuertes RS-FF. Für letzteres sind im Folgenden Funktionstabelle, Schaltsymbol und Synthesetabelle aufgelistet:

| R | S | C | Q <sup>+</sup> |
|---|---|---|----------------|
| - | - | 0 | Q              |
| - | - | 1 | Q              |
| - | - | ↓ | Q              |
| 0 | 0 | ↑ | Q              |
| 0 | 1 | ↑ | 1              |
| 1 | 0 | ↑ | 0              |
| 1 | 1 | ↑ | verboten       |



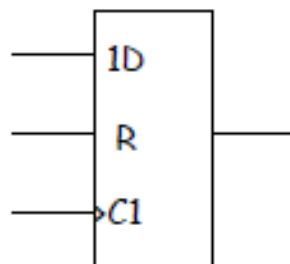
| Q | Q <sup>+</sup> | R | S |
|---|----------------|---|---|
| 0 | 0              | - | 0 |
| 0 | 1              | 0 | 1 |
| 1 | 0              | 1 | 0 |
| 1 | 1              | 0 | - |

### 5.4.6 Flip-Flops mit zusätzlichen Eingängen

Ein zusätzlicher Rücksetzeingang (oder ein Setzeingang) sind auch bei den FF-Typen denkbar, die an sich keine Rücksetz- oder Setz-Funktion mit sich bringen. Es ist dann zu unterscheiden, ob die zusätzliche Funktion taktabhängig (synchron) oder taktunabhängig (asynchron) wirkt. Als Beispiel werden ein D-Flip-Flop mit synchronem und ein D-Flip-Flop mit asynchronem Rücksetzen betrachtet:

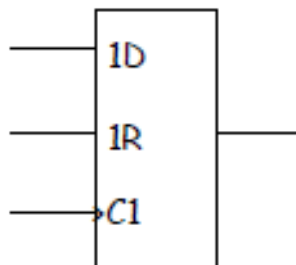
#### D-FF mit asynchronem Reset:

| D | R | C | Q <sup>+</sup> |
|---|---|---|----------------|
| - | 1 | - | 0              |
| - | 0 | 0 | Q              |
| - | 0 | 1 | Q              |
| - | 0 | ↓ | Q              |
| 0 | 0 | ↑ | 0              |
| 1 | 0 | ↑ | 1              |



#### D-FF mit synchronem Reset:

| D | R | C | Q <sup>+</sup> |
|---|---|---|----------------|
| - | - | 0 | Q              |
| - | - | 1 | Q              |
| - | - | ↓ | Q              |
| - | 1 | ↑ | 0              |
| 0 | 0 | ↑ | 0              |
| 1 | 0 | ↑ | 1              |



### 5.4.7 Zusammenfassung

Betrachtet wurden einfache Speicherglieder: die Flip-Flops, die sich entweder im Zustand "gesetzt" (Q=1) oder "rückgesetzt" (Q=0) befinden. Die Zustandsübergänge können mit Funktionen „Rücksetzen“, „Setzen“, „Toggeln“ oder „Daten übernehmen“ kontrolliert werden. „Speichern“ bedeutet: kein Zustandsübergang.

Je nach FF-Typ ist ein Teil dieser Funktionen verfügbar: Betrachtet wurden Flip-Flops mit den Ansteuereingängen RS, D, T und JK.

Ebenso gibt es verschiedene Arten der Taktsteuerung (kein Takt: asynchrones FF; Taktzustandssteuerung, Taktflankensteuerung). Nicht alle FF-Typen sind mit jeder Art der Taktsteuerung möglich. Die folgende Tabelle markiert für die 4 genannten FF-Typen jeweils die mögliche Art der Taktsteuerung:

|                       | RS | D | T | JK |
|-----------------------|----|---|---|----|
| asynchron             | x  | - | - | -  |
| taktzustandsgesteuert | x  | x | - | -  |
| taktflankengesteuert  | x  | x | x | x  |

Im Folgenden sollen Schaltwerke entworfen werden, die für alle Zustandsvariablen taktgesteuerte FFs verwenden, die alle mit demselben Takt beschaltet sind (**synchrone Schaltwerke**).

Mehrere Flip-Flops, die alle mit demselben Takt arbeiten, heißen auch **Register**. Werden darin die Zustandsvariablen eines SWs abgespeichert, so spricht man vom **Zustandsregister** des Schaltwerks.

Wir wollen im Folgenden synchrone Schaltwerke betrachten, die mit einer Taktflankensteuerung arbeiten. Hierfür stehen dann alle 4 betrachteten FF-Typen zur Verfügung.

Wird ein bestimmter FF-Typ beim Entwurf eines Schaltwerks verwendet, so wird für den Entwurfsprozess die Information seiner Synthesetabelle benötigt. Die nachstehende Tabelle fasst diese Informationen für alle 4 betrachteten FF-Typen zusammen:

| Q | Q <sup>+</sup> | R | S | J | K | D | T |
|---|----------------|---|---|---|---|---|---|
| 0 | 0              | - | 0 | 0 | - | 0 | 0 |
| 0 | 1              | 0 | 1 | 1 | - | 1 | 1 |
| 1 | 0              | 1 | 0 | - | 1 | 0 | 1 |
| 1 | 1              | 0 | - | - | 0 | 1 | 0 |

## 5.5 Entwurf von synchronen Schaltwerken

Die notwendigen Entwurfsschritte wurden prinzipiell bereits beim Entwurf des T-Flip-Flops besprochen: man wählt zur Verfügung stehende Speicherglieder aus und entwirft die passenden Ansteuerschaltnetze. Die gleiche Vorgehensweise gilt auch beim Entwurf von komplexen Schaltwerken, die nicht nur eine Zustandsvariable aufweisen. Die Entwurfsschritte lassen sich wie folgt zusammenfassen:

### Entwurfsschritte beim Entwurf synchroner Schaltwerke:

- 1) Definition der Ein- und Ausgabevariablen; Formale Beschreibung des Schaltwerkverhaltens (SW-Graph).
- 2) Festlegen der Zustandskodierung (dabei wird auch die Anzahl der Zustandsvariablen festgelegt); Erstellen der Schaltwerkstabelle.
- 3) Auswahl des Flip-Flop-Typs für jede Zustandsvariable und Erstellen der zugehörigen Ansteuertabelle.
- 4) Entwurf der Ansteuerschaltnetze.
- 5) Entwurf der Ausgabeschaltnetze.
- 6) Zeichnen des Schaltbildes.

### **Beispiel: Geldwechselautomat**

Die Entwurfsschritte 1 – 2 wurden bereits in Abschnitt 5.2 durchgeführt.

Schritt 3: Beide Zustandsvariablen sollen in JK-Flip-Flops gespeichert werden. Die Ansteuertabelle lautet dann:

| $e_1$ | $e_0$ | $q_1$ | $q_0$ | $q_1^+$ | $q_0^+$ | $a_1$ | $a_0$ | $q_1$ | $q_1^+$ | $J_1$ | $K_1$ | $q_0$ | $q_0^+$ | $J_0$ | $K_0$ |
|-------|-------|-------|-------|---------|---------|-------|-------|-------|---------|-------|-------|-------|---------|-------|-------|
| 0     | 0     | 0     | 0     | 0       | 0       | 1     | 1     | 0     | 0       | 0     | -     | 0     | 0       | 0     | -     |
| 0     | 0     | 0     | 1     | 0       | 0       | 1     | 1     | 0     | 0       | 0     | -     | 1     | 0       | -     | 1     |
| 0     | 0     | 1     | 0     | 0       | 0       | 1     | 1     | 1     | 0       | -     | 1     | 0     | 0       | 0     | -     |
| 0     | 0     | 1     | 1     | -       | -       | -     | -     | 1     | -       | -     | -     | 1     | -       | -     | -     |
| 0     | 1     | 0     | 0     | 0       | 1       | 0     | 0     | 0     | 0       | 0     | -     | 0     | 1       | 1     | -     |
| 0     | 1     | 0     | 1     | 1       | 0       | 0     | 0     | 0     | 1       | 1     | -     | 1     | 0       | -     | 1     |
| 0     | 1     | 1     | 0     | 0       | 0       | 1     | 1     | 1     | 0       | -     | 1     | 0     | 0       | 0     | -     |
| 0     | 1     | 1     | 1     | -       | -       | -     | -     | 1     | -       | -     | -     | 1     | -       | -     | -     |
| 1     | 0     | 0     | 0     | 1       | 0       | 0     | 0     | 0     | 1       | 1     | -     | 0     | 0       | 0     | -     |
| 1     | 0     | 0     | 1     | 0       | 0       | 1     | 1     | 0     | 0       | 0     | -     | 1     | 0       | -     | 1     |
| 1     | 0     | 1     | 0     | 0       | 0       | 1     | 1     | 1     | 0       | -     | 1     | 0     | 0       | 0     | -     |
| 1     | 0     | 1     | 1     | -       | -       | -     | -     | 1     | -       | -     | -     | 1     | -       | -     | -     |
| 1     | 1     | 0     | 0     | 0       | 0       | 0     | 0     | 0     | 0       | 0     | -     | 0     | 0       | 0     | -     |
| 1     | 1     | 0     | 1     | 0       | 0       | 0     | 1     | 0     | 0       | 0     | -     | 1     | 0       | -     | 1     |
| 1     | 1     | 1     | 0     | 0       | 0       | 1     | 0     | 1     | 0       | -     | 1     | 0     | 0       | 0     | -     |
| 1     | 1     | 1     | 1     | -       | -       | -     | -     | 1     | -       | -     | -     | 1     | -       | -     | -     |

Schritt 4: Entwurf der Ansteuerschaltnetze $J_1:$ 

|       |               |               |               |               |
|-------|---------------|---------------|---------------|---------------|
|       | $q_0$         |               |               |               |
|       | 0             | 0             | 1             | 0             |
|       | <sub>0</sub>  | <sub>1</sub>  | <sub>5</sub>  | <sub>4</sub>  |
| $q_1$ | -             | -             | -             | -             |
|       | <sub>2</sub>  | <sub>3</sub>  | <sub>7</sub>  | <sub>6</sub>  |
|       | -             | -             | -             | -             |
|       | <sub>10</sub> | <sub>11</sub> | <sub>15</sub> | <sub>14</sub> |
|       | 1             | 0             | 0             | 0             |
|       | <sub>8</sub>  | <sub>9</sub>  | <sub>13</sub> | <sub>12</sub> |
|       | $e_0$         |               |               |               |
|       | $e_1$         |               |               |               |

$$J_1 = \bar{e}_1 \cdot e_0 \cdot q_0 \vee e_1 \cdot \bar{e}_0 \cdot \bar{q}_0$$

$$K_1 = 1$$

 $J_0:$ 

|       |               |               |               |               |
|-------|---------------|---------------|---------------|---------------|
|       | $q_0$         |               |               |               |
|       | 0             | -             | -             | 1             |
|       | <sub>0</sub>  | <sub>1</sub>  | <sub>5</sub>  | <sub>4</sub>  |
| $q_1$ | 0             | -             | -             | 0             |
|       | <sub>2</sub>  | <sub>3</sub>  | <sub>7</sub>  | <sub>6</sub>  |
|       | 0             | -             | -             | 0             |
|       | <sub>10</sub> | <sub>11</sub> | <sub>15</sub> | <sub>14</sub> |
|       | 0             | -             | -             | 0             |
|       | <sub>8</sub>  | <sub>9</sub>  | <sub>13</sub> | <sub>12</sub> |
|       | $e_0$         |               |               |               |
|       | $e_1$         |               |               |               |

$$J_0 = \bar{e}_1 \cdot e_0 \cdot \bar{q}_1$$

$$K_0 = 1$$

Schritt 5: Entwurf der Ausgabeschaltnetze $a_1:$ 

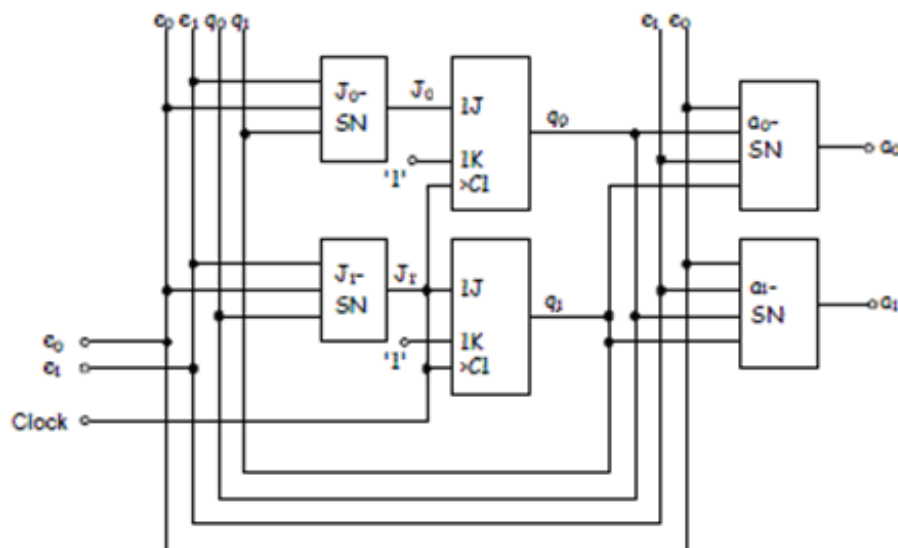
|       |               |               |               |               |
|-------|---------------|---------------|---------------|---------------|
|       | $q_0$         |               |               |               |
|       | 1             | 1             | 0             | 0             |
|       | <sub>0</sub>  | <sub>1</sub>  | <sub>5</sub>  | <sub>4</sub>  |
| $q_1$ | 1             | -             | -             | 1             |
|       | <sub>2</sub>  | <sub>3</sub>  | <sub>7</sub>  | <sub>6</sub>  |
|       | 1             | -             | -             | 0             |
|       | <sub>10</sub> | <sub>11</sub> | <sub>15</sub> | <sub>14</sub> |
|       | 0             | 1             | 1             | 0             |
|       | <sub>8</sub>  | <sub>9</sub>  | <sub>13</sub> | <sub>12</sub> |
|       | $e_0$         |               |               |               |
|       | $e_1$         |               |               |               |

$$a_1 = \bar{e}_1 \cdot \bar{e}_0 \vee \bar{e}_0 \cdot q_1 \vee e_1 \cdot q_0 \vee \bar{e}_1 \cdot q_1$$

 $a_0:$ 

|       |               |               |               |               |
|-------|---------------|---------------|---------------|---------------|
|       | $q_0$         |               |               |               |
|       | 1             | 1             | 0             | 0             |
|       | <sub>0</sub>  | <sub>1</sub>  | <sub>5</sub>  | <sub>4</sub>  |
| $q_1$ | 1             | -             | -             | 1             |
|       | <sub>2</sub>  | <sub>3</sub>  | <sub>7</sub>  | <sub>6</sub>  |
|       | 1             | -             | -             | 1             |
|       | <sub>10</sub> | <sub>11</sub> | <sub>15</sub> | <sub>14</sub> |
|       | 0             | 1             | 0             | 0             |
|       | <sub>8</sub>  | <sub>9</sub>  | <sub>13</sub> | <sub>12</sub> |
|       | $e_0$         |               |               |               |
|       | $e_1$         |               |               |               |

$$a_0 = q_1 \vee \bar{e}_1 \cdot \bar{e}_0 \vee \bar{e}_0 \cdot q_0$$

Schritt 6: Schaltbild zeichnen

Das Schaltbild des SWs ist hier als **Blockschaltbild** angegeben, d. h. die entworfenen Schaltnetze werden nicht detailliert als Gatterschaltung gezeichnet, sondern nur als Block dargestellt. Zu beachten ist allerdings, dass die Blöcke mit den notwendigen Variablen "versorgt" werden.

### Varianten: Ansteuertabelle für D-Flip-Flops bzw. für T-Flip-Flops

| $e_1$ | $e_0$ | $q_1$ | $q_0$ | $q_1^+$ | $q_0^+$ | $q_1$ | $q_1^+$ | $D_1$ | $T_1$ | $q_0$ | $q_0^+$ | $D_0$ | $T_0$ |
|-------|-------|-------|-------|---------|---------|-------|---------|-------|-------|-------|---------|-------|-------|
| 0     | 0     | 0     | 0     | 0       | 0       | 0     | 0       | 0     | 0     | 0     | 0       | 0     | 0     |
| 0     | 0     | 0     | 1     | 0       | 0       | 0     | 0       | 0     | 0     | 1     | 0       | 0     | 1     |
| 0     | 0     | 1     | 0     | 0       | 0       | 1     | 0       | 0     | 1     | 0     | 0       | 0     | 0     |
| 0     | 0     | 1     | 1     | -       | -       | 1     | -       | -     | -     | 1     | -       | -     | -     |
| 0     | 1     | 0     | 0     | 0       | 1       | 0     | 0       | 0     | 0     | 0     | 1       | 1     | 1     |
| 0     | 1     | 0     | 1     | 1       | 0       | 0     | 1       | 1     | 1     | 1     | 0       | 0     | 1     |
| 0     | 1     | 1     | 0     | 0       | 0       | 1     | 0       | 0     | 1     | 0     | 0       | 0     | 0     |
| 0     | 1     | 1     | 1     | -       | -       | 1     | -       | -     | -     | 1     | -       | -     | -     |
| 1     | 0     | 0     | 0     | 1       | 0       | 0     | 1       | 1     | 1     | 0     | 0       | 0     | 0     |
| 1     | 0     | 0     | 1     | 0       | 0       | 0     | 0       | 0     | 0     | 1     | 0       | 0     | 1     |
| 1     | 0     | 1     | 0     | 0       | 0       | 1     | 0       | 0     | 1     | 0     | 0       | 0     | 0     |
| 1     | 0     | 1     | 1     | -       | -       | 1     | -       | -     | -     | 1     | -       | -     | -     |
| 1     | 1     | 0     | 0     | 0       | 0       | 0     | 0       | 0     | 0     | 0     | 0       | 0     | 0     |
| 1     | 1     | 0     | 1     | 0       | 0       | 0     | 0       | 0     | 0     | 1     | 0       | 0     | 1     |
| 1     | 1     | 1     | 0     | 0       | 0       | 1     | 0       | 0     | 1     | 0     | 0       | 0     | 0     |
| 1     | 1     | 1     | 1     | -       | -       | 1     | -       | -     | -     | 1     | -       | -     | -     |

Die Entwurfsschritte 4-6 für diese beiden Varianten können als Übung durchgeführt werden. Ein weiteres Beispiel für einen Schaltwerksentwurf wird im nächsten Abschnitt behandelt.

## 5.6 Schaltwerksstrukturen

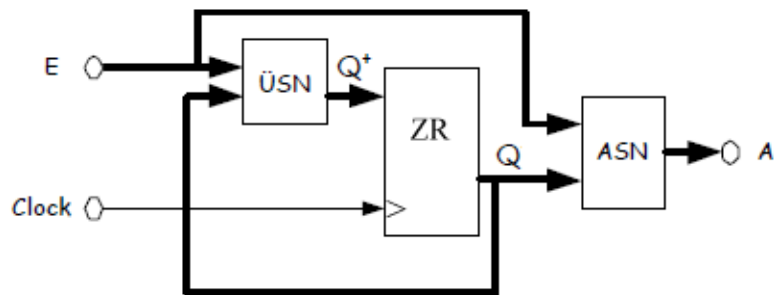
Eine Digitalschaltung, die ein Schaltwerk realisiert, besteht also in der Regel aus drei Teilen: den Speichergliedern, den Schaltnetzen für die Ansteuerung der Speicherglieder und im Allgemeinen den Schaltnetzen für die Ausgabe.

Diese drei Blöcke werden auch wie folgt bezeichnet:

- Zustandsregister (ZR; engl.: state register SR)
- Überführungsschaltnetz (ÜSN; engl.: next state decoder NSD)
- Ausgabeschaltnetz (ASN; engl.: output decoder OD).

Bei den Begriffen Zustandsregister bzw. Überführungsschaltznetz ist noch keine Festlegung auf einen bestimmten FF-Typ erfolgt: Das ÜSN sorgt dafür, dass bei der nächsten aktiven Taktflanke der nächste Zustand (Folgezustand; engl. next state) in das ZR übernommen wird.

Im allgemeinsten Fall hängt die Ausgabe direkt von der Eingabe ab. Dann liegt eine Mealy-Struktur vor. Das Blockschaltbild des Mealy-Schaltwerks kann wie folgt dargestellt werden:

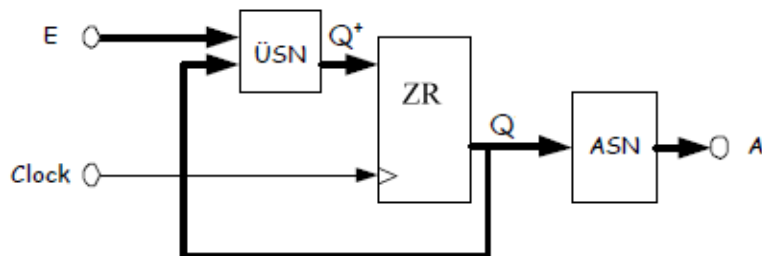


Mealy-Schaltwerk

$$Q^+ = f(E, Q)$$

$$A = f(E, Q)$$

Beim Moore-Schaltwerk fehlt der Pfad von der Eingabe zum ASN. Die Ausgabe hängt also nur von Zustand ab:

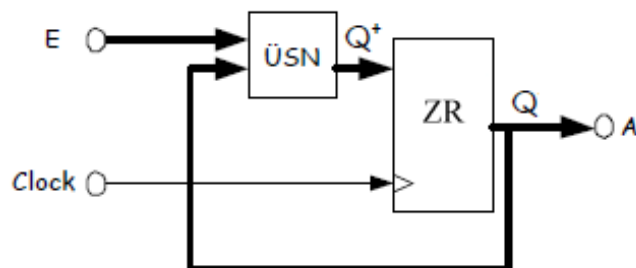


Moore-Schaltwerk

$$Q^+ = f(E, Q)$$

$$A = f(Q)$$

Wenn die Ausgabe direkt von den Zustandsvariablen abgeleitet werden kann, kann das ASN ganz entfallen. Dann liegt ein Medwedew-Schaltwerk vor:



Medwedew-Schaltwerk

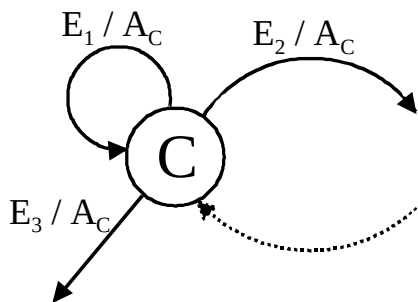
$$Q^+ = f(E, Q)$$

$$A = Q$$

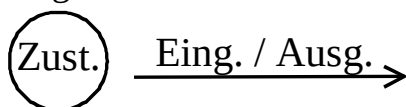
## Vereinfachte Darstellung des Graphen eines Moore-SW:

Hängt die Ausgabe nur von Zustand ab (Moore-SW), so braucht die Ausgabe nicht an jeder Kante notiert werden, die von diesem Zustand ausgehen. Vielmehr kann sie im Zustandsknoten selbst notiert werden:

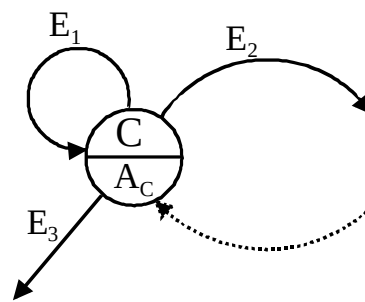
ausführliche Darstellung:



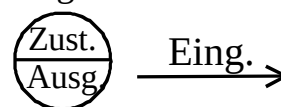
Legende:



vereinfachte Darstellung:



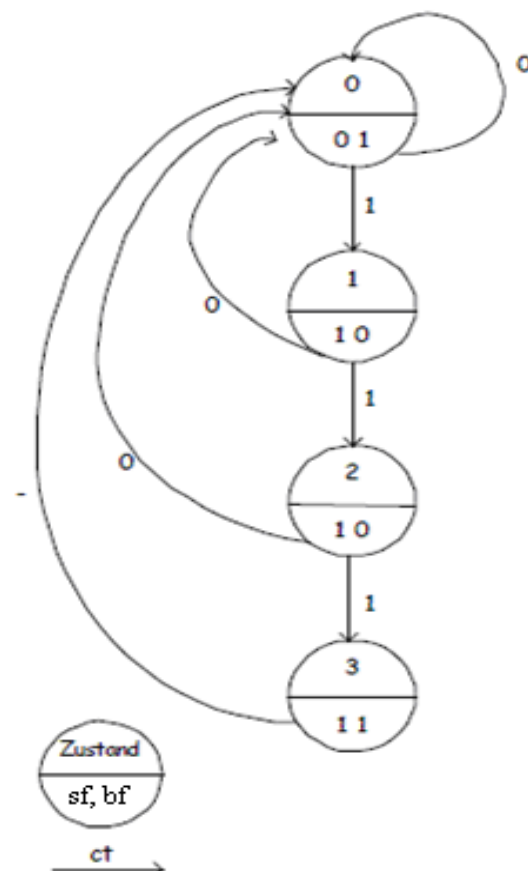
Legende:



## Weiteres Entwurfsbeispiel:

Zu entwerfen sei ein Moore-Steuerwerk, dessen SW-Graph wie neben stehend gegeben ist:

Schritt 1: Eine Eingangsvariable *ct*; zwei Ausgangsvariablen *sf* und *bf* (bereits in der Aufgabenstellung spezifiziert).





Schritt 2:

Zustandskodierung: Es wird die minimal erforderliche Anzahl von zwei Zustandsvariablen festgelegt und eine binäre Zustandskodierung gewählt (d. h. die Kodierung der fortlaufend bezeichneten Zustände entspricht dem Dualcode).

| Zustand | $q_1$ | $q_0$ |
|---------|-------|-------|
| 0       | 0     | 0     |
| 1       | 0     | 1     |
| 2       | 1     | 0     |
| 3       | 1     | 1     |

## SW-Tabelle:

| ct | $q_1$ | $q_0$ | $q_1^+$ | $q_0^+$ | sf | bf |
|----|-------|-------|---------|---------|----|----|
| 0  | 0     | 0     | 0       | 0       | 0  | 1  |
| 0  | 0     | 1     | 0       | 0       | 1  | 0  |
| 0  | 1     | 0     | 0       | 0       | 1  | 0  |
| 0  | 1     | 1     | 0       | 0       | 1  | 1  |
| 1  | 0     | 0     | 0       | 1       | 0  | 1  |
| 1  | 0     | 1     | 1       | 0       | 1  | 0  |
| 1  | 1     | 0     | 1       | 1       | 1  | 0  |
| 1  | 1     | 1     | 0       | 0       | 1  | 1  |

Schritt 3:

Die Zustandsvariablen sollen in D-Flip-Flops abgespeichert werden. Dann lautet die Ansteuertabelle:

| ct | $q_1$ | $q_0$ | $D_1$ | $D_0$ |
|----|-------|-------|-------|-------|
| 0  | 0     | 0     | 0     | 0     |
| 0  | 0     | 1     | 0     | 0     |
| 0  | 1     | 0     | 0     | 0     |
| 0  | 1     | 1     | 0     | 0     |
| 1  | 0     | 0     | 0     | 1     |
| 1  | 0     | 1     | 1     | 0     |
| 1  | 1     | 0     | 1     | 1     |
| 1  | 1     | 1     | 0     | 0     |

Schritt 4:

Entwurf der Ansteuerfunktionen:

 $D_1$ :

|       |              |              |              |              |
|-------|--------------|--------------|--------------|--------------|
|       | $q_0$        |              |              |              |
|       | 0            | 0            | 1            | 0            |
|       | <sub>0</sub> | <sub>1</sub> | <sub>5</sub> | <sub>4</sub> |
| $q_1$ | 0            | 0            | 0            | 1            |
|       | <sub>2</sub> | <sub>3</sub> | <sub>7</sub> | <sub>6</sub> |
|       | $ct$         |              |              |              |

$$D_1 = ct \cdot \overline{q_1} q_0 \vee ct \cdot q_1 \cdot \overline{q_0}$$

$$= ct \cdot (q_1 \oplus q_0)$$

 $D_0$ :

|       |              |              |              |              |
|-------|--------------|--------------|--------------|--------------|
|       | $q_0$        |              |              |              |
|       | 0            | 0            | 0            | 1            |
|       | <sub>0</sub> | <sub>1</sub> | <sub>5</sub> | <sub>4</sub> |
| $q_1$ | 0            | 0            | 0            | 1            |
|       | <sub>2</sub> | <sub>3</sub> | <sub>7</sub> | <sub>6</sub> |
|       | $ct$         |              |              |              |

$$D_0 = ct \cdot \overline{q_0}$$

Schritt 5:

Entwurf der Ausgabefunktionen:

sf:

|       |              |              |              |              |
|-------|--------------|--------------|--------------|--------------|
|       | $q_0$        |              |              |              |
|       | 0            | 1            | 1            | 0            |
|       | <sub>0</sub> | <sub>1</sub> | <sub>5</sub> | <sub>4</sub> |
| $q_1$ | 1            | 1            | 1            | 1            |
|       | <sub>2</sub> | <sub>3</sub> | <sub>7</sub> | <sub>6</sub> |
|       | $ct$         |              |              |              |

$$sf = q_1 \vee q_0$$

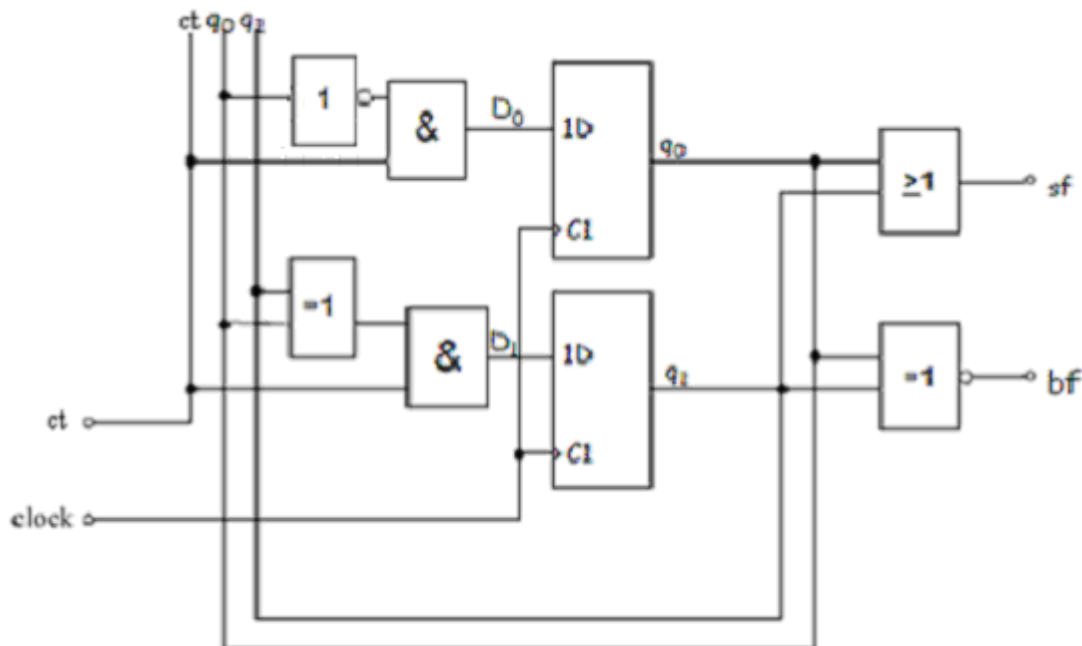
bf:

|       |              |              |              |              |
|-------|--------------|--------------|--------------|--------------|
|       | $q_0$        |              |              |              |
|       | 1            | 0            | 0            | 1            |
|       | <sub>0</sub> | <sub>1</sub> | <sub>5</sub> | <sub>4</sub> |
| $q_1$ | 0            | 1            | 1            | 0            |
|       | <sub>2</sub> | <sub>3</sub> | <sub>7</sub> | <sub>6</sub> |
|       | $ct$         |              |              |              |

$$bf = q_1 \cdot q_0 \vee \overline{q_1} \cdot \overline{q_0} = q_1 \oplus q_0$$

Schritt 6:

Zeichnen des Schaltbilds:



## 5.7 Spezielle Schaltwerke

## 5.7.1 Zähler

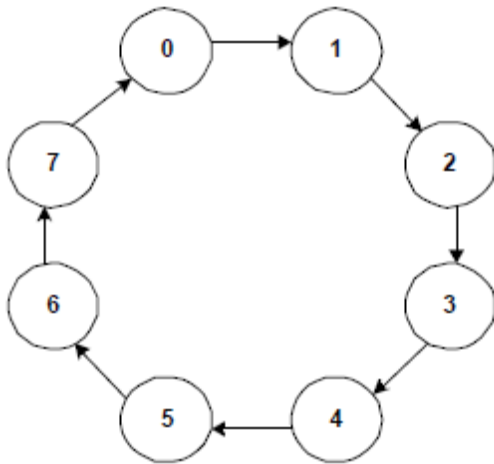
Ein Zähler ist ein Medwedew-Schaltwerk, das bei jedem Takt den Zustand in der Weise wechselt, dass alle Zustände nacheinander erreicht werden und dann dieser Ablauf wiederholt wird.

Bezeichnungen:

Modulo-m-Zähler: Zähler mit  $m$  Zuständen, in der Regel nummeriert von 0 bis  $m-1$ .

n-Bit-Zähler: Zähler mit  $2^n$  Zuständen, in der Regel nummeriert von 0 bis  $2^n - 1$ .

Der Zustandsgraph eines 3-Bit-Zählers (Modulo-8-Zählers) ist dann der folgende:



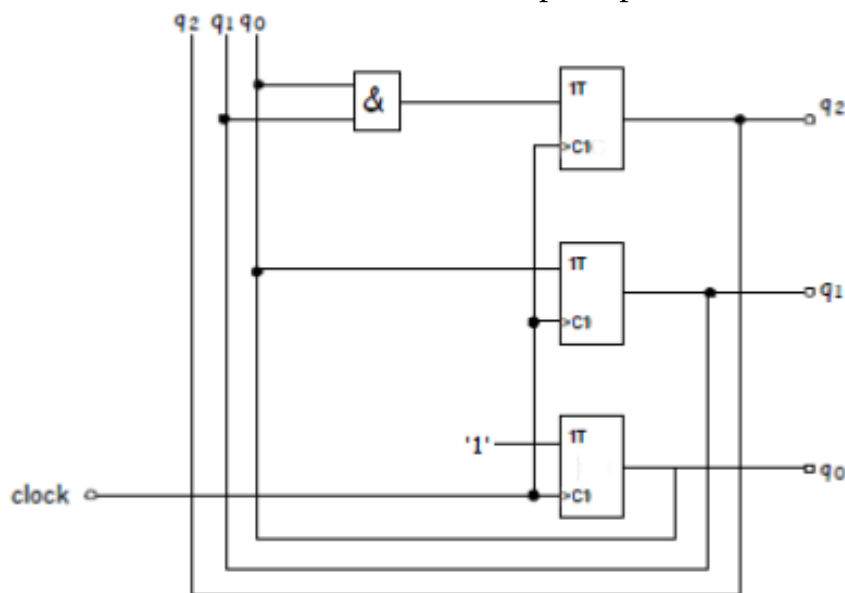
Außer dem Takt ist – falls nur die Grundfunktion realisiert wird – kein weiteres Eingangssignal vorhanden.

Die Zustandsvariablen dienen auch als Ausgabevariablen (Medwedew-SW).

Werden die Zustände binär kodiert, liefert das SW die Zählfolge:

000 – 001 – 010 – 011 – 100 – 101 – 110 – 111

Der Entwurf dieses Zählers mit T-Flip-Flops führt auf die folgende Schaltung:



Bei Erweiterung auf 4 Bit (entsprechend auch auf beliebig viele Bits erweiterbar) gelten folgende Gleichungen für die Ansteuerfunktionen:

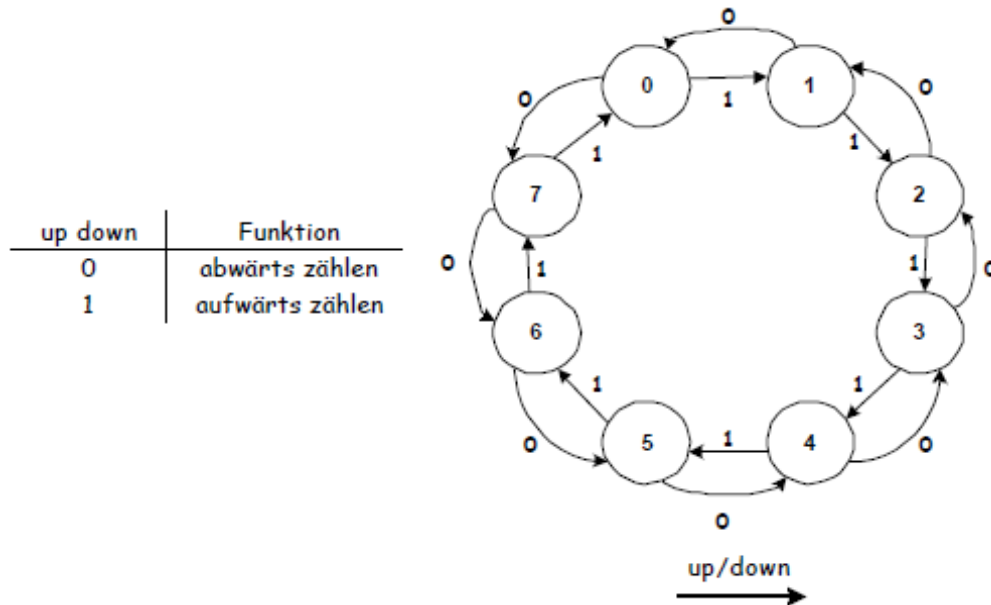
$$T_0 = 1$$

$$T_1 = q_0$$

$$T_2 = q_0 \cdot q_1$$

$$T_3 = q_0 \cdot q_1 \cdot q_2$$

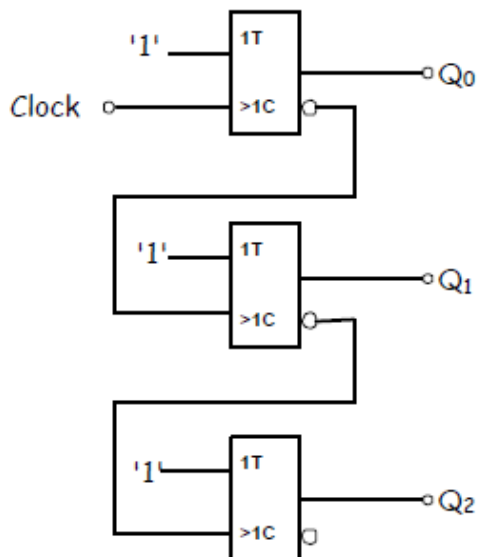
In der Regel gibt es aber weitere Eingangssignale, z. Bsp. für das Umschalten zwischen Vorwärts- und Rückwärtszählen (oder aufwärts/abwärts; engl. up/down):



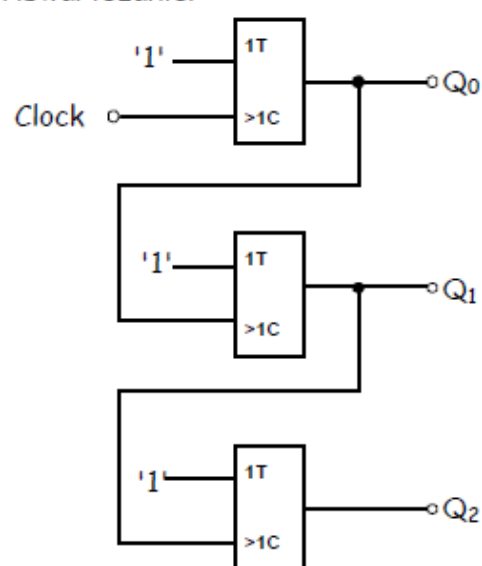
### Asynchrone Zähler:

Ein Aufwärts- bzw. Abwärtszähler lässt sich auch mit weniger Aufwand realisieren: Die beiden folgenden Schaltungen verwenden ausschließlich T-Flip-Flops:

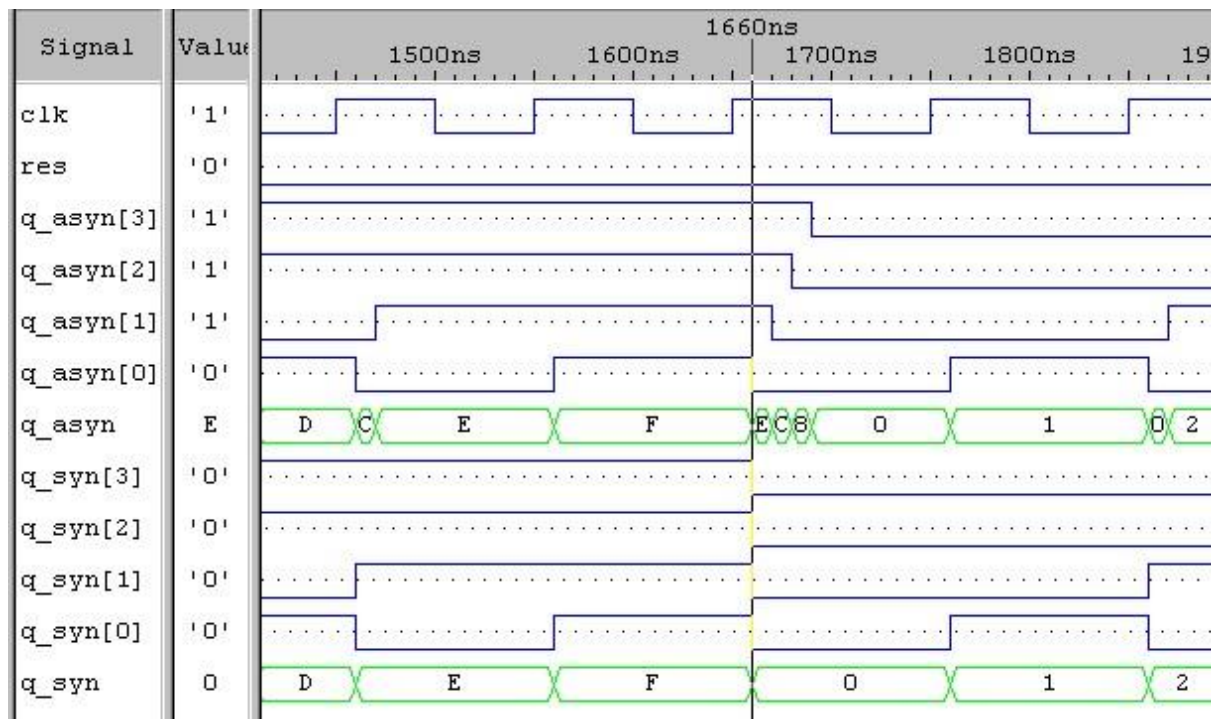
*Aufwärtszähler:*



*Abwärtszähler:*



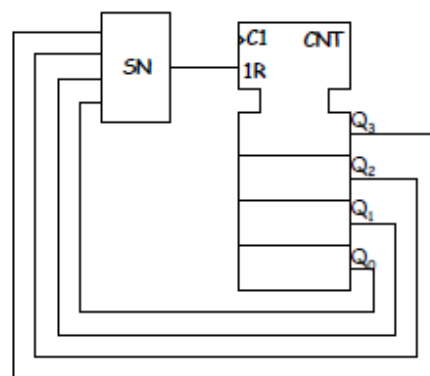
Obwohl alle Zustandsvariablen in Flip-Flops abgespeichert sind, sind dies keine synchronen Schaltungen, da jedes FF einen anderen Takt hat: Es handelt sich um asynchrone Zähler (engl. ripple counter). Der Nachteil ist, dass die Ausgänge nicht gleichzeitig umschalten. Dies verdeutlicht nachfolgende Simulation eines synchronen und eines asynchronen 4-Bit-Zählers:



Eine wichtige Anwendung von asynchronen Zählern, bei der dieser Nachteil keine Rolle spielt, ist der Einsatz als Frequenzteiler.

### Realisierung von Modulo-m-Zählern mit Standardbausteinen:

Die Vorgehensweise zeigt die folgende Realisierung eines Modulo-10-Zählers mit Hilfe eines gängigen 4-Bit-Zählers, der als zusätzlichen Eingang noch einen synchronen Reset-Eingang aufweist:



$$\text{reset} = q_3 \cdot q_2 \cdot q_1 \cdot q_0$$

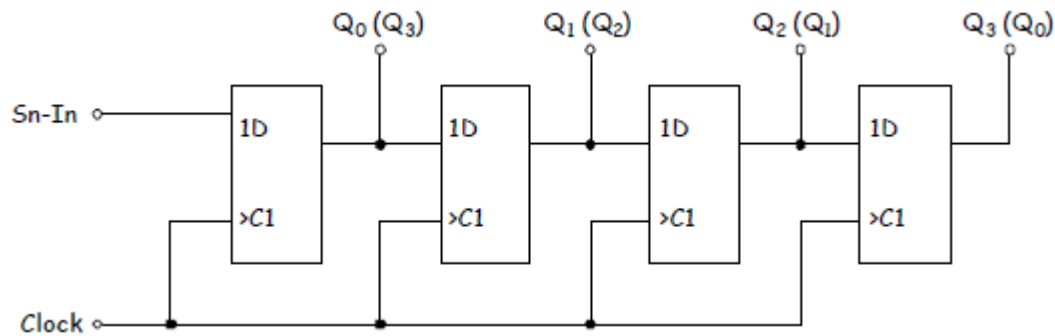
(Zust.9 wird decodiert)

Übliche Zusatzeingänge bei Standard-Zählern sind:

- reset (synchron oder asynchron)
- up\_down
- enable (bzw. hold)
- load (zusätzliche Dateneingänge, synchrones oder asynchrones Laden)

## 5.7.2 Schieberegister

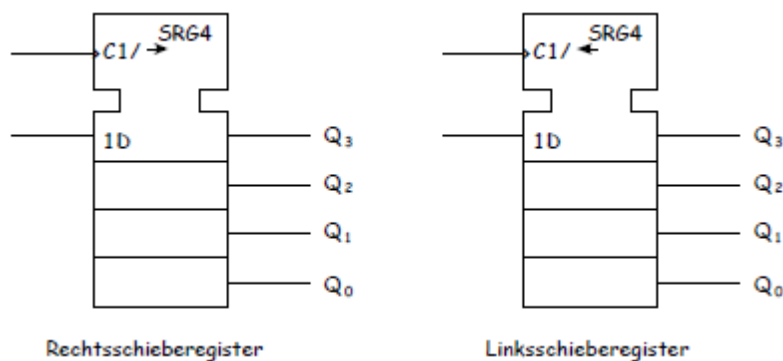
Die folgende Schaltung realisiert ein so genanntes Schieberegister: je nach Benennung der Ausgänge werden die Funktionen „Linksschieben“ bzw. „Rechtsschieben“ implementiert:



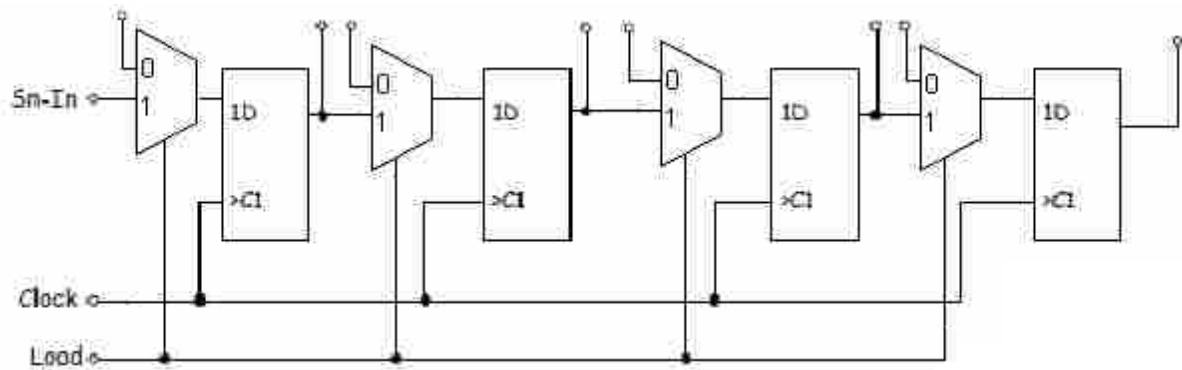
Rechtsschieben  
 (6) 0 1 1 0  $\xrightarrow{\text{SerIn} = 0}$  0 0 1 1 (3)      Division durch 2

Linksschieben  
 (6) 0 1 1 0  $\xrightarrow{\text{SerIn} = 0}$  1 1 0 0 (12)      Multiplikation mit 2

Das Schaltsymbol lautet wie folgt:

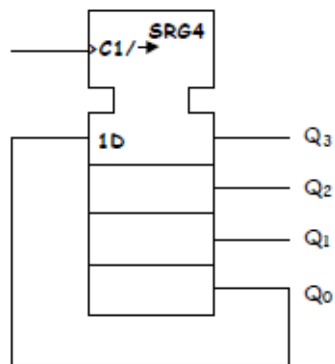


Eine weitere Anwendung des Schieberegisters ist die Parallel-Wandlung serieller Daten. Für den umgekehrten Weg (Parallel-Seriell-Wandlung von Daten) ist ein ladbares Schieberegister (z. Bsp. Baustein 74LS194A: ladbares 4 – Bit – Links – Rechts – Schieberegister) erforderlich, das prinzipiell wie folgt mit Hilfe von 2:1-Multiplexern aufgebaut werden kann:



Schieberegister, bei denen der Ausgang am Ende der Kette wieder auf den Eingang zurückgeführt wird, arbeiten wie Zähler, wobei die Länge der Zählfolge von den am Anfang geladenen Daten abhängt.

### Ringschieberegister (Umlaufregister)

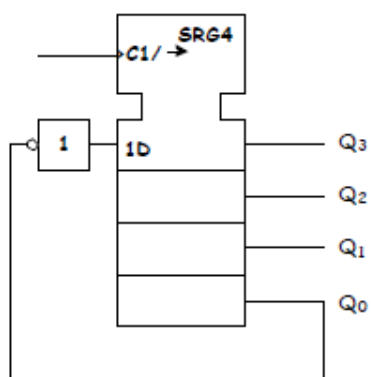


|     |   |   |   |   |
|-----|---|---|---|---|
| (0) | 1 | 1 | 0 | 1 |
| (1) | 1 | 1 | 1 | 0 |
| (2) | 0 | 1 | 1 | 1 |
| (3) | 1 | 0 | 1 | 1 |
| (0) | 1 | 1 | 0 | 1 |

Modulo-4-Zähler

### Johnson-Zähler = switched - tail - counter

Bsp.:



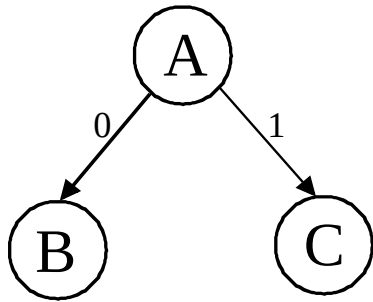
|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 0 | 1 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

Modulo-8-Zähler

## 5.8 Weiterführende Themen

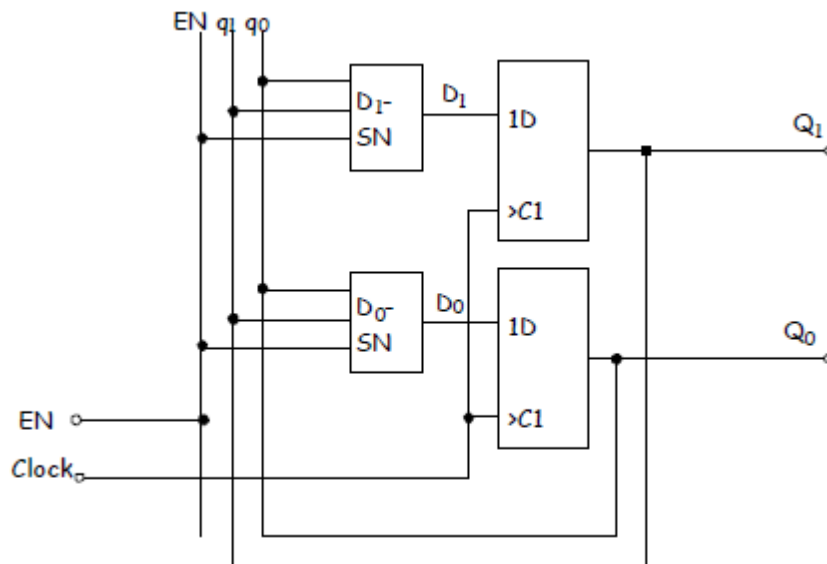
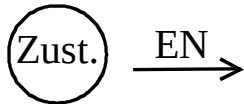
### 5.8.1 Synchronisation der Eingabevariablen

Betrachtet wird der folgende Auszug aus einem SW-Graphen, die Tabelle der Zustandskodierung und das Blockschaltbild des SWs:



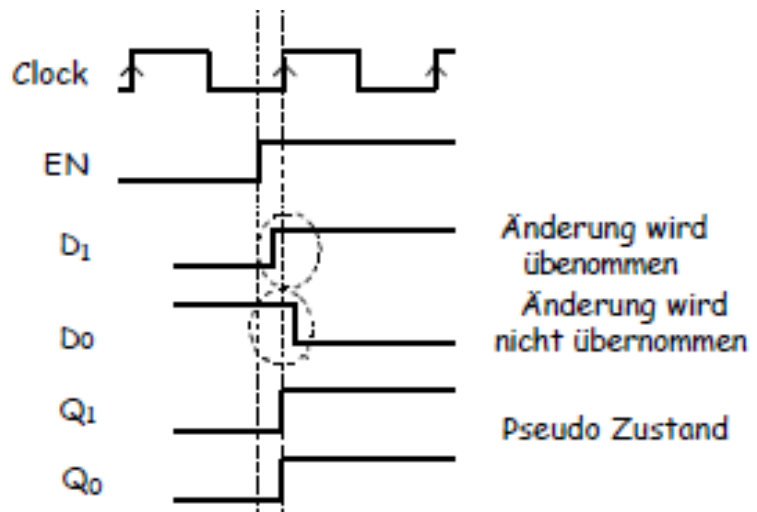
| Zustand              | Q <sub>1</sub> | Q <sub>0</sub> |
|----------------------|----------------|----------------|
| A                    | 0              | 0              |
| B                    | 0              | 1              |
| C                    | 1              | 0              |
| <i>Pseudozustand</i> | 1              | 1              |

Legende:

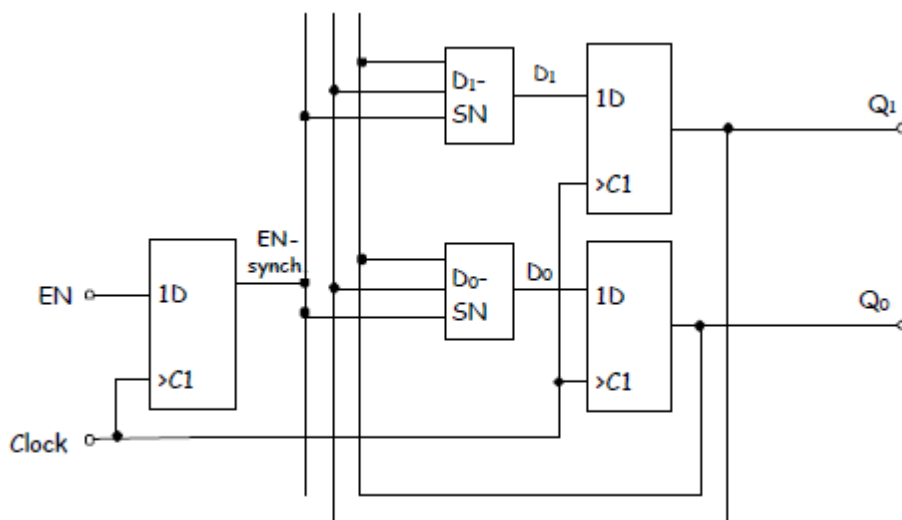




Erreicht das SW den Zustand A und ändert sich – erst kurz vor der nächsten aktiven Taktflanke – das Eingangssignal  $EN$ , so kann auf Grund von unterschiedlichen Laufzeiten in den Ansteuerschaltnetzen das SW in einen undefinierten Zustand übergehen:



Diese Problematik lässt sich vermeiden, wenn man die Eingangsvariablen synchronisiert: Die Änderungen am Eingang der Ansteuerschaltnetze erfolgen dann immer kurz nach einer aktiven Taktflanke und damit mit maximalem Abstand zur nächsten aktiven Taktflanke. Damit ist die betrachtete Fehlfunktion ausgeschlossen. Ein Nachteil – neben dem zusätzlichen Aufwand – ist, dass die Reaktion des SWs um einen Takt verzögert erfolgt. Die nächste Abbildung zeigt die Synchronisation der Eingabevariable  $EN$ :



### 5.8.2 Kodierung nach Ausgabe

Bei Moore-Schaltwerken kann man versuchen, die Zustände so zu kodieren, dass Zustandsvariablen direkt als Ausgabevariablen dienen können (Kodierung nach Ausgabe). Dadurch kann man den Aufwand beim ASN reduzieren. Gelingt dies für alle Ausgabevariablen, so entsteht eine Medwedew-Struktur ohne ASN.

Im Beispiel aus 5.6 könnte man statt der binären Zustandskodierung eine Alternative wählen, bei der zumindest ein Ausgang ( $bf$ ) identisch mit einer Zustandsvariablen ( $q_0$ ) ist.

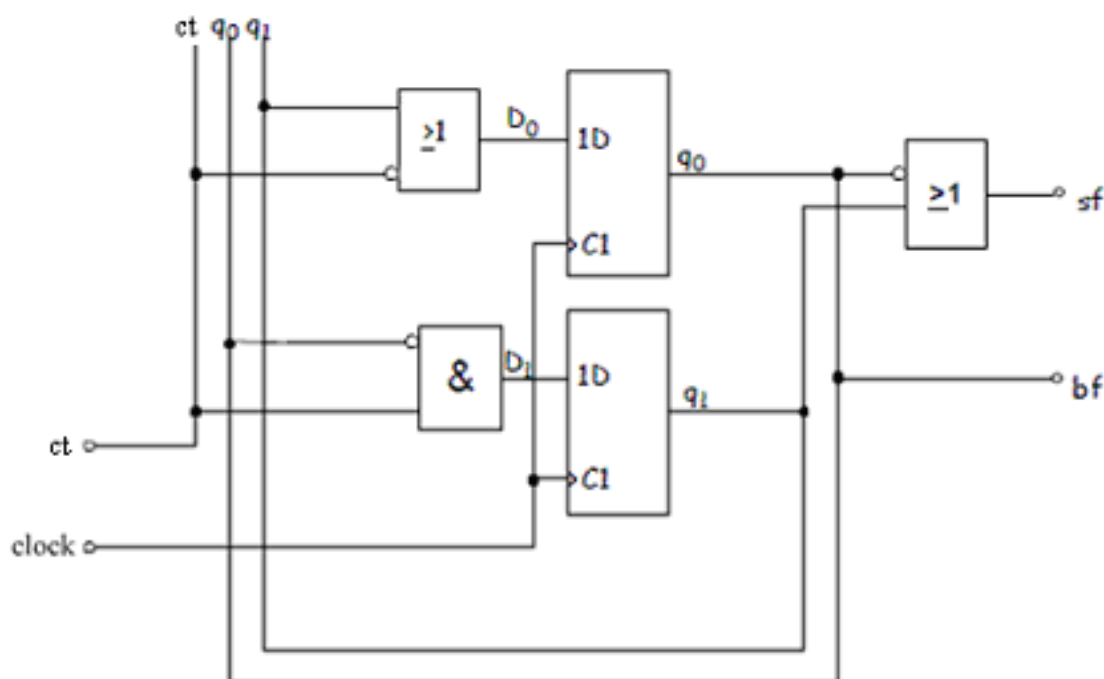
| Zustand | $q_1$ $q_0$ | $sf$ $bf$ |
|---------|-------------|-----------|
| 0       | 0 1         | 0 1       |
| 1       | 0 0         | 1 0       |
| 2       | 1 0         | 1 0       |
| 3       | 1 1         | 1 1       |

(Kodierung nach Ausgabe; Variante 1)

Dann ergibt sich folgende Schaltwerks- und Ansteuertabelle:

| ct | $q_1$ | $q_0$ | $q_1^+$ | $q_0^+$ | $sf$ | $bf$ | $D_1$ | $D_0$ |
|----|-------|-------|---------|---------|------|------|-------|-------|
| 0  | 0     | 0     | 0       | 1       | 1    | 0    | 0     | 1     |
| 0  | 0     | 1     | 0       | 1       | 0    | 1    | 0     | 1     |
| 0  | 1     | 0     | 0       | 1       | 1    | 0    | 0     | 1     |
| 0  | 1     | 1     | 0       | 1       | 1    | 1    | 0     | 1     |
| 1  | 0     | 0     | 1       | 0       | 1    | 0    | 1     | 0     |
| 1  | 0     | 1     | 0       | 0       | 0    | 1    | 0     | 0     |
| 1  | 1     | 0     | 1       | 1       | 1    | 0    | 1     | 1     |
| 1  | 1     | 1     | 0       | 1       | 1    | 1    | 0     | 1     |

Nach dem Entwurf der Schaltnetze ( $D_0 = q_1 \vee \overline{ct}$ ;  $D_1 = ct \cdot \overline{q_0}$ ;  $sf = q_1 \vee \overline{q_0}$ ;  $bf = q_0$ ) erhält man das folgende Schaltbild:

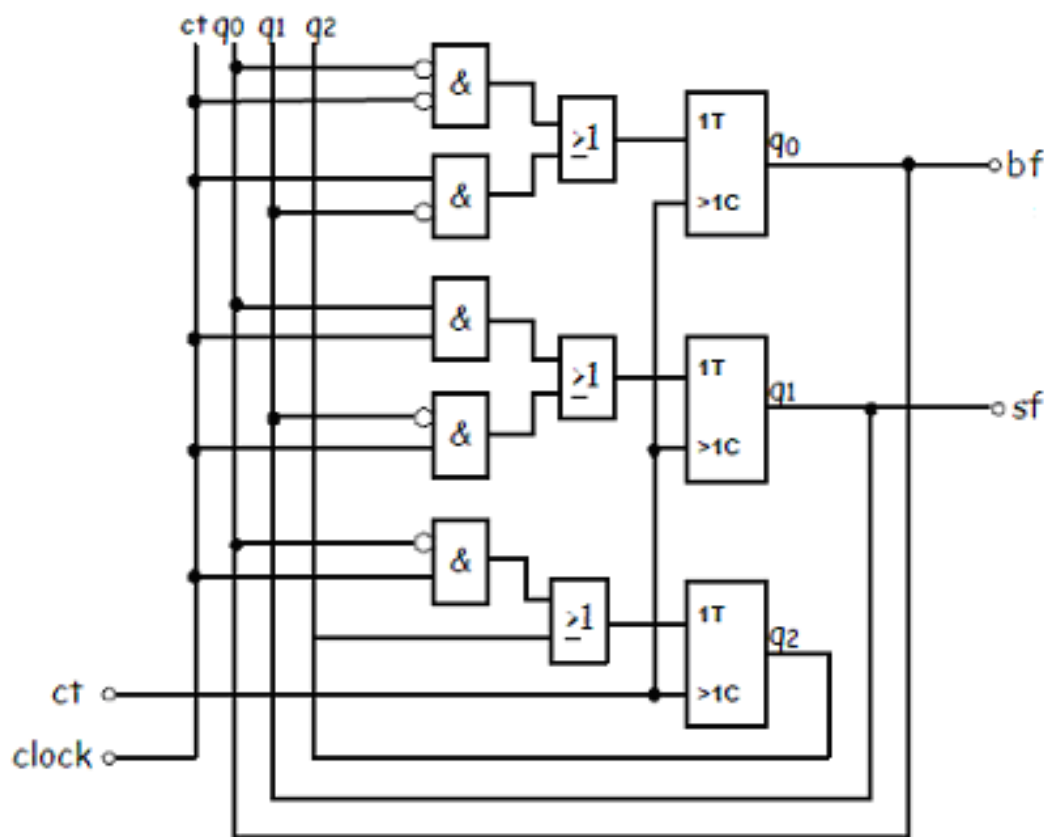


Führt man eine dritte Zustandsvariable ein, so gelingt es, beide Ausgänge direkt aus Zustandsvariablen abzuleiten:

| Zustand | $q_2$ | $q_1$ | $q_0$ | $sf$ | $bf$ |
|---------|-------|-------|-------|------|------|
| 0       | 0     | 0     | 1     | 0    | 1    |
| 1       | 0     | 1     | 0     | 1    | 0    |
| 2       | 1     | 1     | 0     | 1    | 0    |
| 3       | 0     | 1     | 1     | 1    | 1    |

(Kodierung nach Ausgabe; Variante 2)

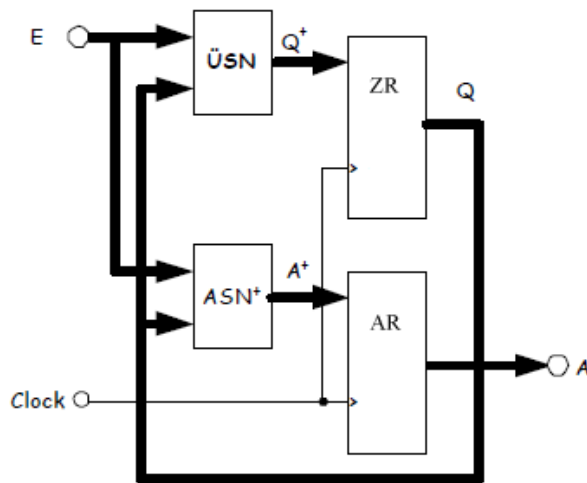
Der Entwurf mit T-Flip-Flops (Übungsaufgabe!) liefert das folgende Ergebnis:



In dieser Schaltung werden alle Ausgangsvariablen direkt aus FF-Ausgängen abgeleitet: Die Ausgangsvariablen ändern sich gleichzeitig und zum gleichen Zeitpunkt wie die (restlichen) Zustandsvariablen. Es gibt keine strukturelle Unterscheidung mehr zwischen Zustands- und Ausgabevariablen.

### 5.8.3 Moore-Struktur mit Ausgaberegister

Eine SW-Struktur, bei der kein struktureller Unterschied zwischen Zustands- und Ausgabevariablen besteht, ist bei Moore-Schaltwerken immer möglich: Da die Ausgabe nur vom Zustand abhängt, können Folgezustand ( $Q^+$ ) und Ausgabe im Folgezustand ( $A^+$ ) gleichzeitig ermittelt werden:



AR = Ausgaberegister

$ASN^+ = SN$  für Ausgabe im Folgezustand:  $A^+ = f(Q^+) = f(E, Q)$

Beim Entwurf eines Moore-SW mit Ausgaberegistern werden Spalten für die Ausgabe im Folgezustand in die Schaltwerkstabelle aufgenommen. Der Entwurf wird vereinfacht, wenn man D-Flip-Flops für das Ausgaberegister verwendet: Dann werden von den Ausgaberegistern keine Rückführungen benötigt. Die Spalten für die Ausgabe im aktuellen Zustand können bei der Schaltwerkstabelle entfallen.

Das soll an dem bereits bekannten Beispiel (SW-Graph in 5.6) verdeutlicht werden. Da auch bei dieser Struktur eine Kodierung nach Ausgabe Sinn macht (weil jetzt ein Teil der  $ASN^+$ -Schaltnetze entfallen kann), wird die Kodierung aus 5.7.2 (Variante 1) verwendet:

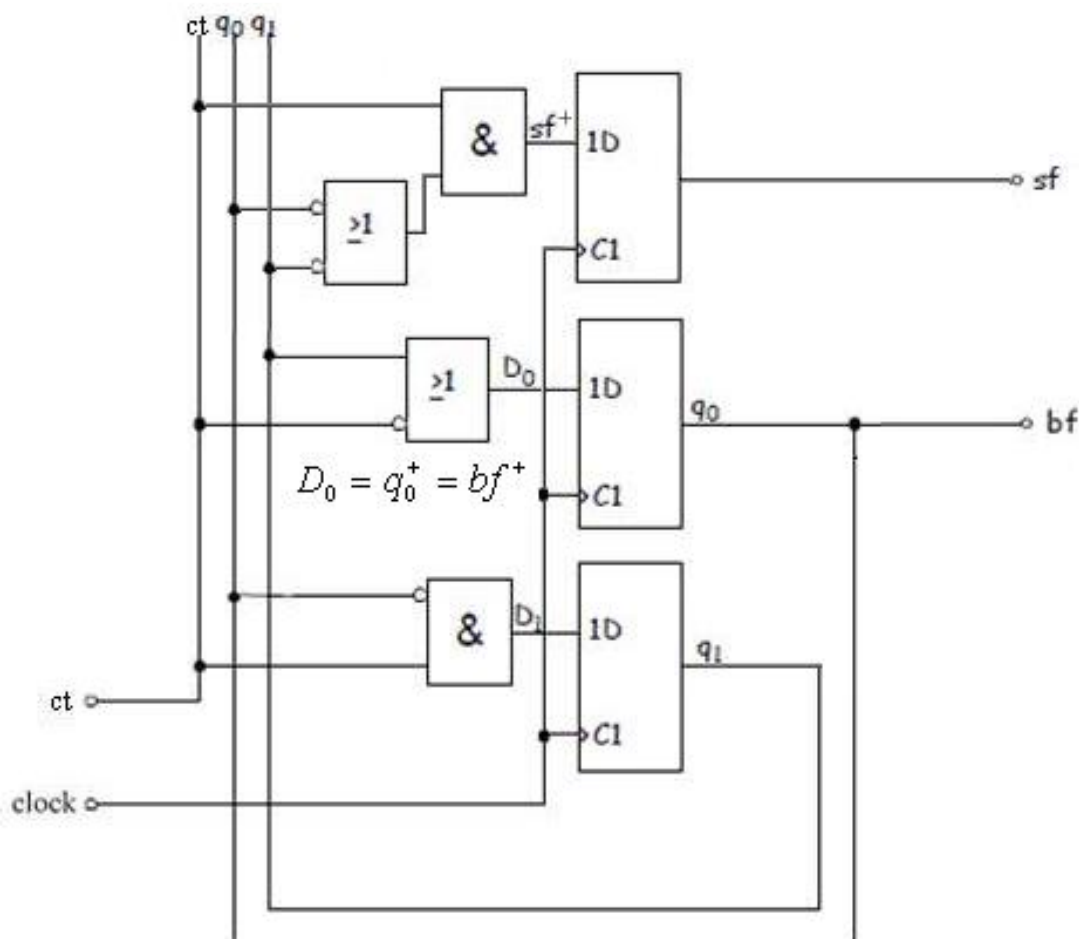
Zustandskodierung und Schaltwerks-/Ansteuertabelle:

| Zustand | $q_1$ $q_0$ | ct | $q_1$ | $q_0$ | $q_1^+$ | $q_0^+$ | $sf^+$ | $bf^+$ | $D_1$ | $D_0$ |
|---------|-------------|----|-------|-------|---------|---------|--------|--------|-------|-------|
| 0       | 0 1         | 0  | 0     | 0     | 0       | 1       | 0      | 1      | 0     | 1     |
| 1       | 0 0         | 0  | 0     | 1     | 0       | 1       | 0      | 1      | 0     | 1     |
| 2       | 1 0         | 0  | 1     | 0     | 0       | 1       | 0      | 1      | 0     | 1     |
| 3       | 1 1         | 0  | 1     | 1     | 0       | 1       | 0      | 1      | 0     | 1     |
|         |             | 1  | 0     | 0     | 1       | 0       | 1      | 0      | 1     | 0     |
|         |             | 1  | 0     | 1     | 0       | 0       | 1      | 0      | 0     | 0     |
|         |             | 1  | 1     | 0     | 1       | 1       | 1      | 1      | 1     | 1     |
|         |             | 1  | 1     | 1     | 0       | 1       | 0      | 1      | 0     | 1     |

Die Ansteuerfunktionen sind die selben wie in 5.7.2:  $D_0 = q_1 \vee \overline{ct}$ ;  $D_1 = ct \cdot \overline{q_0}$ .

Für die Ausgabe im Folgezustand ergibt sich:  $bf^+ = q_0^+$  (wegen Kodierung nach Ausgabe) und:  $sf^+ = ct \cdot \overline{q_0} \vee ct \cdot \overline{q_1} = ct \cdot (\overline{q_0} \vee \overline{q_1})$  (man beachte die Abhängigkeit von  $ct$ ).

Damit lautet das Schaltbild dieser Moore-Variante mit Ausgaberegister:



Alle Ausgabevariablen ändern sich taktsynchron und ohne Verzögerung durch ein Ausgabeschaltnetz (schnellere Reaktion). Dafür erfordert diese Variante einen zusätzlichen Hardware-Aufwand (Ausgaberegister; A<sup>+</sup>SN aufwendiger als „normales“ ASN).